Linux for Absolute Beginners

A Step-by-Step Practical Guide for MLOps Exams

Starts from zero. Every command comes with a plain-English explanation, a real terminal screenshot, and a line-by-line reading of what happened.

Based on the course reference *Linux Basic Commands: A Practical Reference for MLOps Coursework* (Dr. Santosh Kumar).

About the screenshots: all were captured from a real Ubuntu Linux shell. Three screenshots are labelled **SAMPLE OUTPUT** (nvidia-smi, ssh, scp) because they need a GPU or a remote server that this machine did not have. Your own screen will differ slightly in names, dates and numbers. That is normal.

Contents

Start Here: What Is Linux and How Do I Open It? 4

How to get a Linux terminal . . . 4

Reading the prompt . . . 4

Anatomy of every command . . . 5

Seven golden rules for beginners . . . 5

Step 1: File and Directory Navigation 6

pwd : Print Working Directory (where am I?) . . . 6

ls : List Directory Contents (what is here?) . . . 6

cd : Change Directory (move around) . . . 8

The Linux folder map . . . 10

find : search for files . . . 11

which and whereis : where is this command? . . . 11

Step 2: File and Directory Management 13

mkdir : Make Directory . . . 13

touch : create an empty file or update its timestamp . . . 14

Putting text into a file with echo and > (needed a lot in exams) . . . 14

cp : Copy files or directories . . . 14

mv : Move or Rename . . . 15

rm : Remove files and directories . . . 15

Step 3: Viewing and Editing File Contents 17

cat : show a whole file . . . 17

less (and more) : page through a big file . . . 17

head and tail : first or last lines . . . 18

nano and vim : edit files in the terminal . . . 18

grep : search text inside files . . . 19

Step 4: File Permissions and Ownership 21

chmod : change permissions . . . 21

chown : change owner . . . 22

sudo : run one command as administrator . . . 22

Step 5: Process and Job Management 24

ps : list processes . . . 24

top and htop : live monitor . . . 24

kill and kill -9 : stop a process . . . 25

& and nohup : run jobs in the background . . . 26

jobs, fg, bg : manage background jobs . . . 27

Step 6: System Monitoring and Resource Usage 28

df : free disk space . . . 28

du : how big is this folder? . . . 28
free : memory (RAM) usage . . . 28
nvidia-smi : GPU status . . . 29
watch : repeat a command automatically . . . 29
Step 7: Networking and Remote Access **31**
ssh : log in to another computer . . . 31
scp : copy files over ssh . . . 31
curl and wget : download files or call an API . . . 31
Checking which program is using a port . . . 32
Step 8: Archiving, Compression and Transfer **34**
tar : bundle files into one archive . . . 34
zip and unzip . . . 34
rsync : smart synchronisation . . . 35
Step 9: Text Processing and Piping **36**
| : the pipe . . . 36
> and >> : redirection . . . 36
wc : count lines, words, characters . . . 36
sort and uniq . . . 36
awk and sed . . . 37
Step 10: Environment Variables, Python and Packages **38**
Python virtual environment and pip . . . 38
Step 11: Git Basics for MLOps **40**
git clone, pull, push . . . 43
Step 12: Useful MLOps Command Patterns **44**
Run a training job in the background with a dated log . . . 44
tmux : a terminal session that never dies . . . 44
Find and clean up large checkpoint files . . . 45
Check that a model service is up . . . 45
Step 13: Practice Exam. Do It Yourself **47**
Worked solution . . . 47
Cheat Sheet: Every Command on One Page **49**
Common errors and what they mean . . . 50
Exam day checklist . . . 50

Start Here: What Is Linux and How Do I Open It?

Linux is an operating system, like Windows or macOS. The difference is that in MLOps (machine learning in production) almost every server, cloud machine and Docker container runs Linux. You control it by typing text commands instead of clicking icons. The place where you type is called the **terminal**. The program inside the terminal that understands your commands is called the **shell** (usually one called **bash**).

Do not worry if it looks scary. A command is just a short word that tells the computer to do one job: `ls` means 'show me the files', `cd` means 'go to a folder'. In this guide every command is run for real on a Linux machine, and the screenshot shows exactly what you should see on your own screen.

How to get a Linux terminal

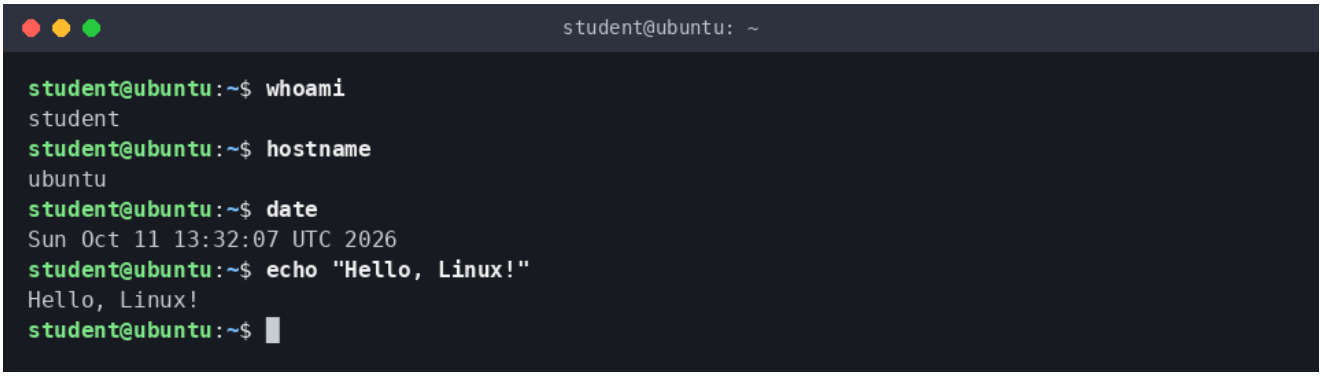
Your computer	What to do
Exam / college lab (Ubuntu)	Press Ctrl + Alt + T together. A black or purple window opens. That is the terminal. Or click 'Show Applications' (9 dots) and search for 'Terminal'.
Windows 10 / 11 (practice at home)	Open the Start menu, type PowerShell, right-click it, choose 'Run as administrator'. Type: wsl --install then press Enter, restart the PC, and open the new 'Ubuntu' app. Create a username and password when it asks.
macOS	Press Cmd + Space, type Terminal, press Enter. macOS is not Linux but nearly all commands in this guide work the same.
No installation possible	Use a free online Linux terminal in your browser (search 'online Ubuntu terminal'), or use Google Colab and put an exclamation mark before each command, e.g. !ls

> **TIP:** Typing a password in the terminal shows NOTHING on the screen, not even dots. That is normal. Type it and press Enter.

Reading the prompt

When the terminal is ready it shows a line that ends with $. This is the **prompt**. It means 'I am waiting for your command'. You type after the $ and press **Enter** to run it. You never type the prompt itself, only the command.

Screenshot 1: Your very first commands.



What happened:

- **student@ubuntu:~$** is the prompt. **student** = your user name, **ubuntu** = the computer name, **~** = the folder you are in (~ means your Home folder), **$** = ready for a command.
- **whoami** prints your user name. **hostname** prints the computer's name. **date** prints date and time.
- **echo** simply prints whatever text you give it. Text with spaces goes inside quotes.

Anatomy of every command

Almost every Linux command follows the same pattern:

`command -options arguments`

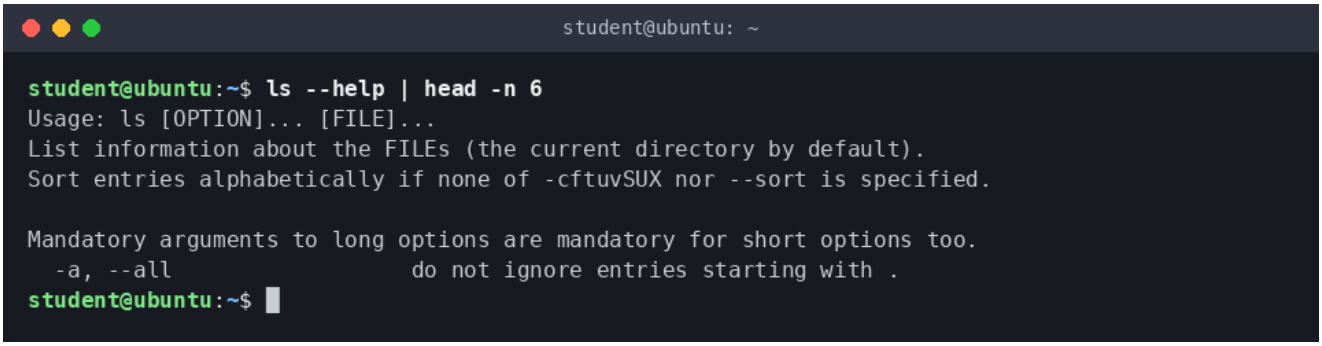
- **command**: what to do (ls, cp, mkdir ...).
- **options** (also called flags): start with a dash. They change how the command behaves, e.g. `ls -l`. You can join them: `ls -lah` is the same as `ls -l -a -h`.
- **arguments**: what to do it on, usually a file or folder name, e.g. `cp a.txt b.txt`.

Seven golden rules for beginners

- **Linux is case sensitive.** `Documents` and `documents` are different names.
- **Spaces separate words.** A file called 'my file.txt' must be written as `"my file.txt"` in quotes. Better: never use spaces in file names, use `my_file.txt`.
- **Press the Tab key** to auto-complete names. Type `cd Doc`, press Tab, and it finishes 'Documents'. Saves time and typos.
- **Press the Up arrow** to bring back your previous command. Press it again to go further back.
- **Ctrl + C** stops a running command. Use it when something hangs or runs forever.
- **Ctrl + L** (or typing `clear`) cleans the screen.
- **No news is good news.** If a command prints nothing and returns to the prompt, it usually worked.

> **TIP:** Forgot how a command works? Type `command --help` (short help) or `man command` (full manual, press q to quit).

Screenshot 2: Asking a command for help.



What happened:

- The **|** (pipe) sends the long help text into **head -n 6**, which shows only the first 6 lines. You will learn both soon.

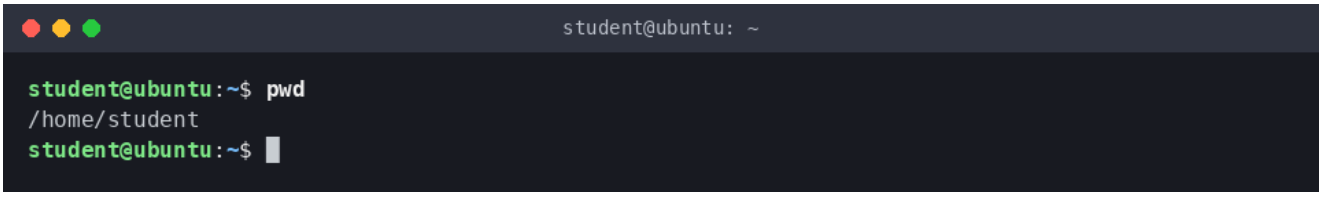
Step 1: File and Directory Navigation

A **directory** is just a folder. Linux organises everything as a tree of folders. You are always 'standing' inside one folder, called the **current working directory**. Navigation commands tell you where you are and move you around.

pwd : Print Working Directory (where am I?)

Prints the full path of the folder you are in. A **path** is the address of a file, with folders separated by /. A path that starts with / is **absolute** (full address from the very top). Check this before running scripts that use relative paths.

Screenshot 3: pwd shows the full address of the current folder.



What happened:

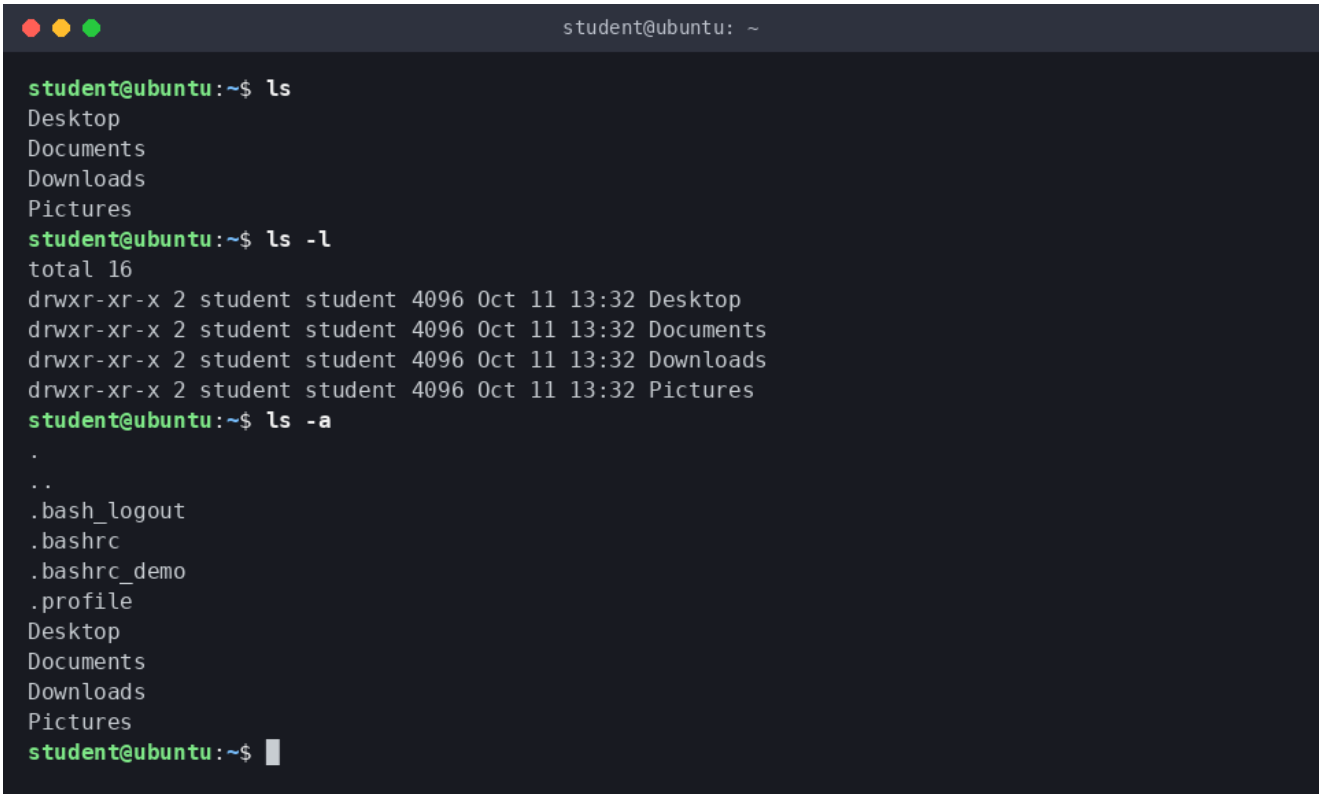
- **/home/student** is the Home folder of the user 'student'. The prompt shows it as **~**.

ls : List Directory Contents (what is here?)

Shows the files and folders inside the current folder. Useful flags:

Flag	Meaning
-l	long format: permissions, owner, size, date
-a	all: also show hidden files (names starting with a dot)
-h	human-readable sizes (KB, MB, GB). Use with -l
-t	sort by time, newest first
-R	recursive: also list inside sub-folders

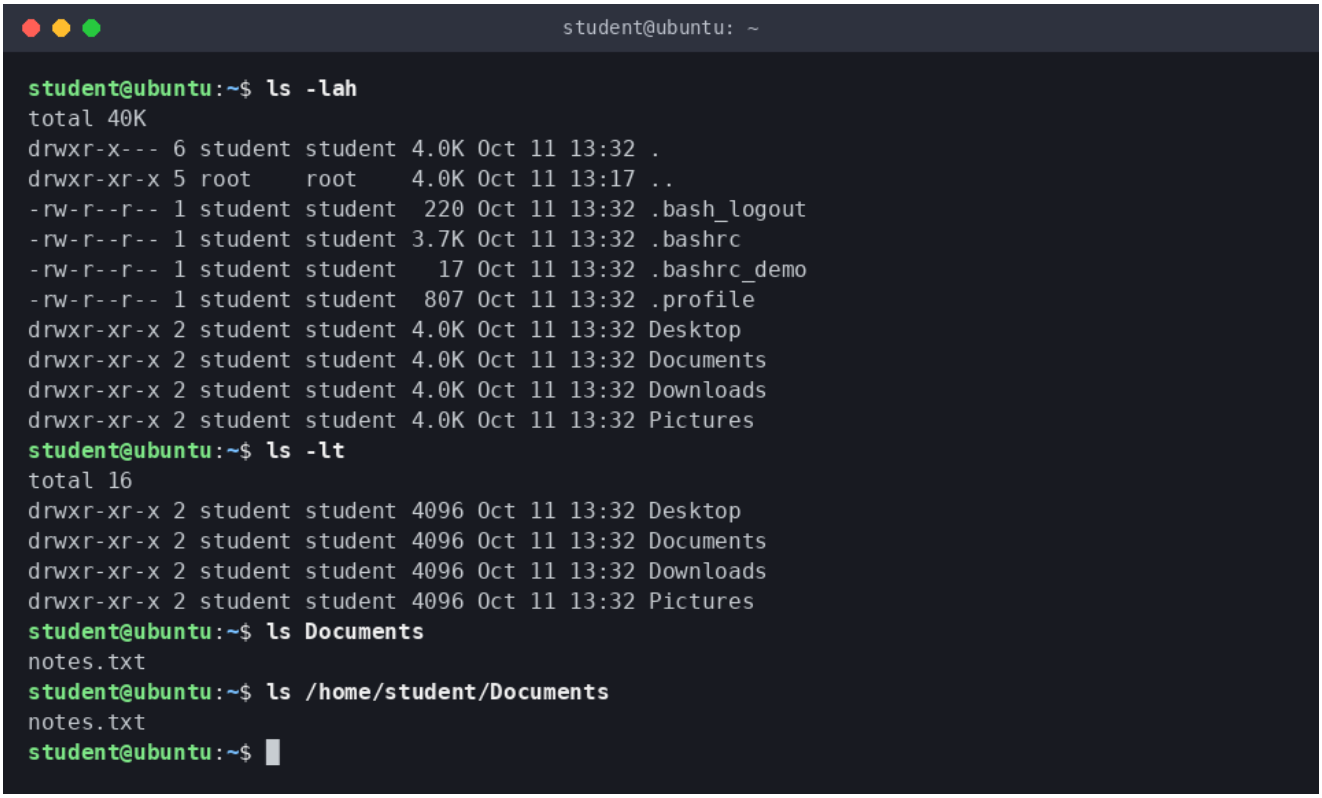
Screenshot 4: ls in three flavours: plain, long (-l) and with hidden files (-a).



What happened:

- Plain **ls** shows only names.
- **ls -l** shows details. First letter 'd' means directory. Next you see permissions, owner, group, size in bytes, date and name.
- **ls -a** shows hidden files too, such as **.bashrc_demo**. The single dot **.** means 'this folder' and **..** means 'the folder above'.

Screenshot 5: Combined flags (-lah), sorting by time (-lt) and listing another folder.



What happened:

- **ls -lah** = long + all + human-readable. This is the most used form.
- You can give **ls** a folder name to list that folder without moving into it. Relative path (Documents) and absolute path (/home/student/Documents) give the same result.

cd : Change Directory (move around)

Moves you into a different folder. Special shortcuts:

Command	Meaning
cd foldername	go into a folder inside the current one
cd ..	go up one level (to the parent folder)
cd ~ or cd	go to your Home folder
cd -	go back to the previous folder you were in
cd /	go to the root (very top) of the whole system

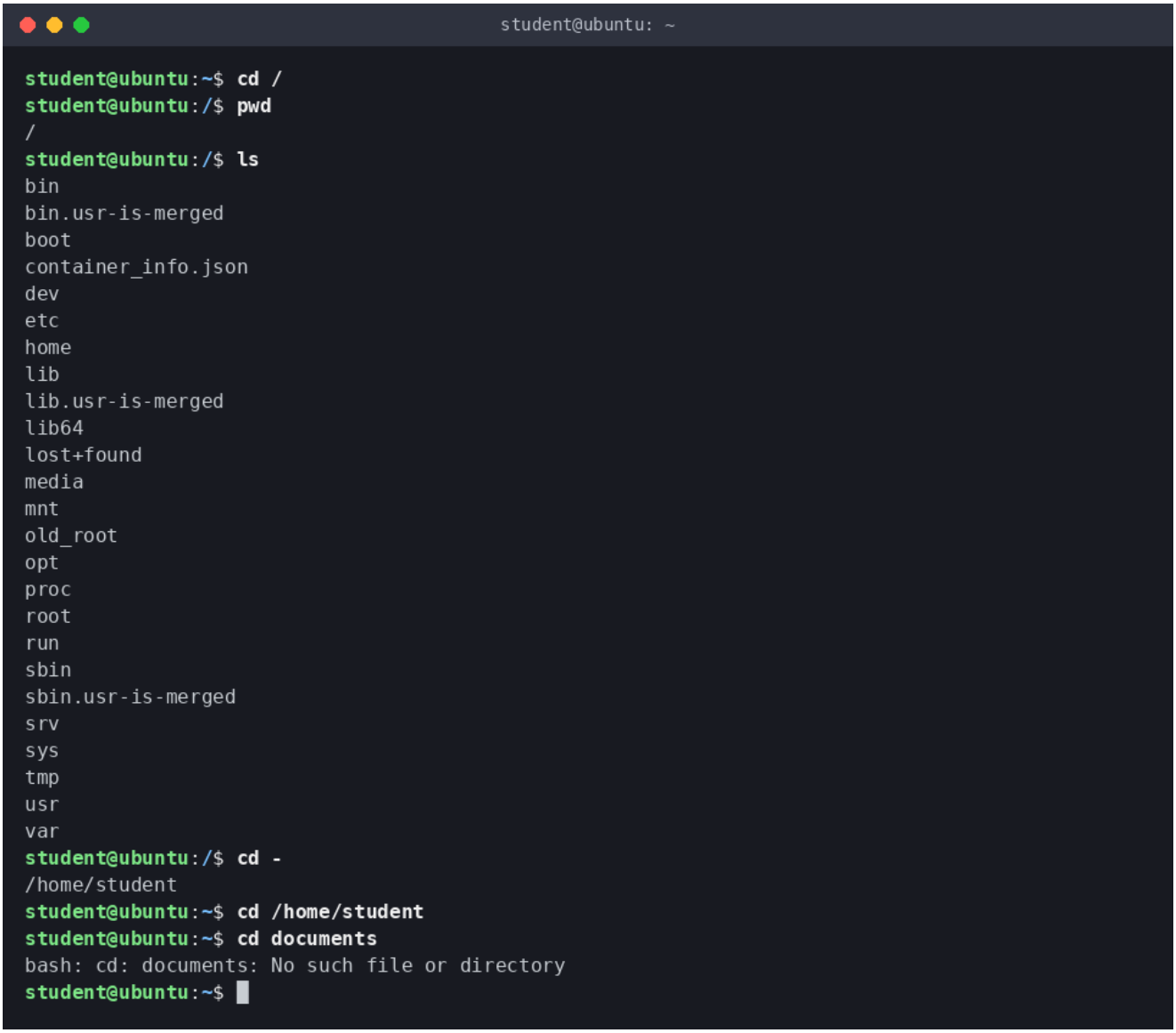
Screenshot 6: Moving down into folders and back up.



What happened:

- Notice the prompt changes: **~/Documents**, then back to **~**. The prompt always shows where you are.
- **cd ..** climbs one level. **cd ~** jumps Home from anywhere.

Screenshot 7: Jumping to the root, returning with 'cd -', and a typical beginner error.



What happened:

- **cd -** took us back to where we were before (the Home folder).

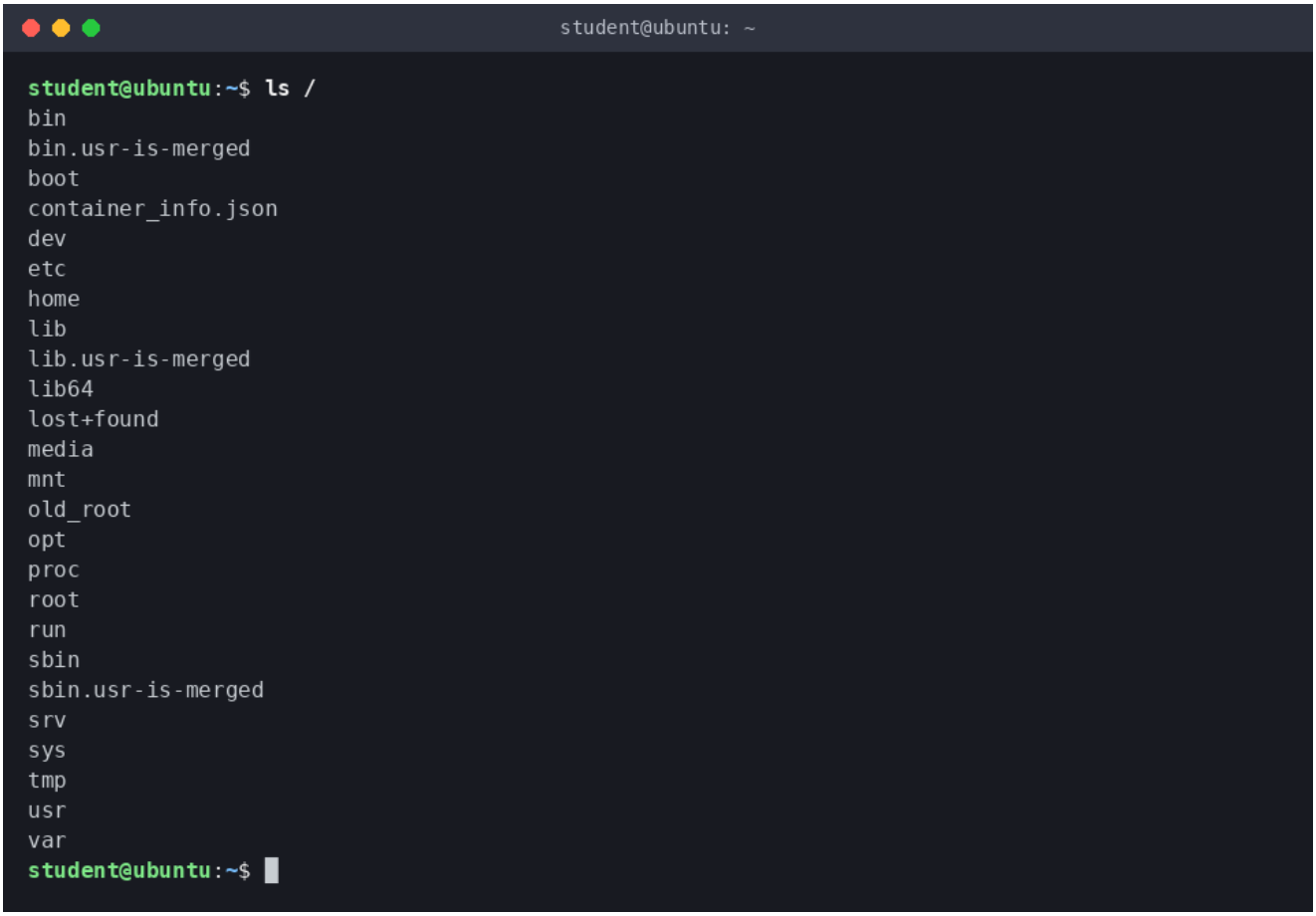
- The last command failed: **No such file or directory**, because we typed lowercase **documents**. Linux is case sensitive, so it must be **Documents**. Read the error message, it tells you what is wrong.

> **EXAM HINT:** In exams you are often asked: 'Show your current directory and list all files including hidden ones.' Answer: pwd then ls -la. Take a screenshot of both.

The Linux folder map

Everything on Linux starts at the root `/`. Here is what is directly under it:

Screenshot 8: The top-level folders of a Linux system.



What happened:

- Your list may differ slightly. Items such as **container_info.json** and **old_root** exist only on the demo machine used to make this guide.

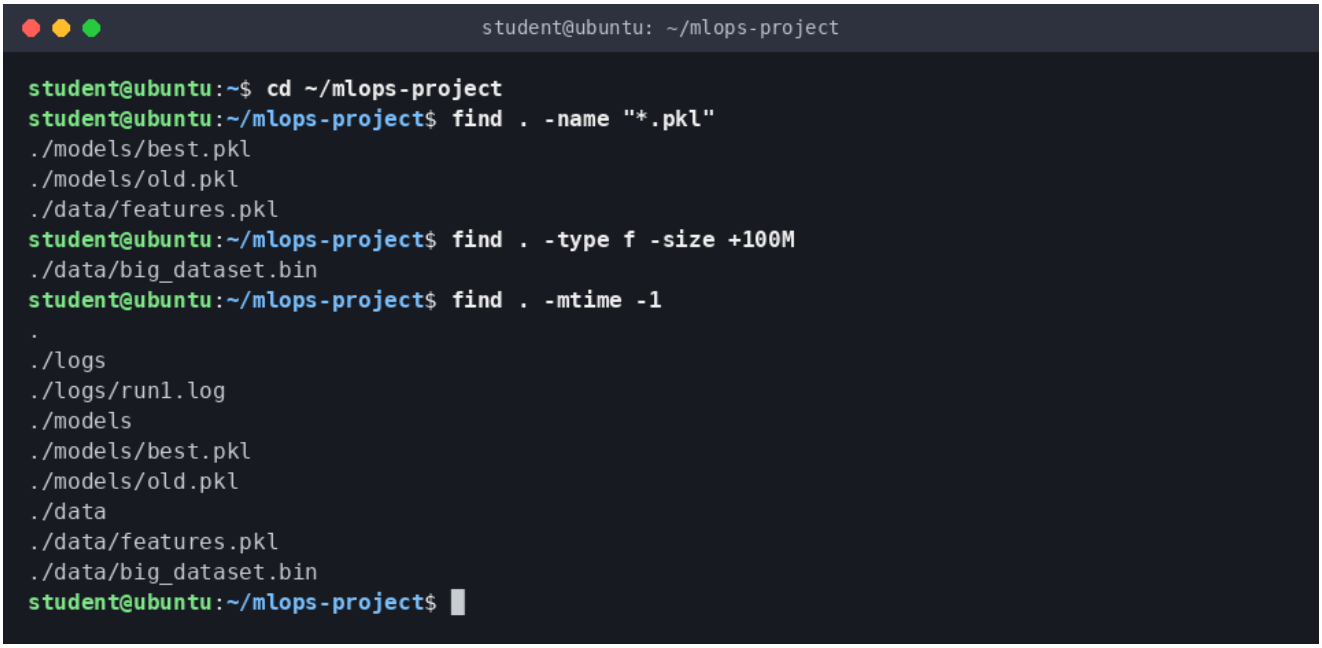
Folder	What lives there
/home	personal folders of users (your files go here)
/root	home folder of the administrator (root)
/etc	configuration files of the system and programs
/var	changing data such as logs (/var/log)
/tmp	temporary files, cleaned automatically
/usr, /bin	installed programs and commands
/mnt, /media	attached disks and USB drives (datasets often mounted here)

Folder	What lives there
/dev, /proc, /sys	special folders that represent hardware and running processes

find : search for files

Searches a folder and everything below it for files matching a name, type, size or age. Very useful for finding stray checkpoint files, logs or configs. We first create a few files so there is something to find.

Screenshot 9: find by name, by size and by modification time.



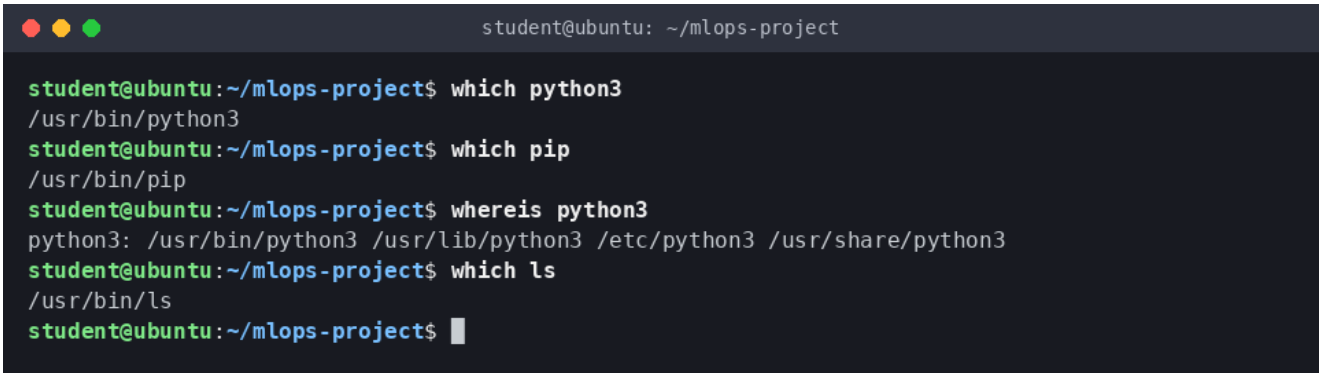
What happened:

- **.** means 'start searching here'. **-name "*.pkl"** matches every file ending in .pkl; ***** is a wildcard that means 'anything'. Always put the pattern in quotes.
- **-type f** = only files (use **d** for directories). **-size +100M** = bigger than 100 megabytes.
- **-mtime -1** = modified within the last 1 day.

which and whereis : where is this command?

which shows the exact program that would run when you type a command. Handy to check which Python or pip is active. **whereis** also shows manual pages and source locations.

Screenshot 10: Locating programs.



What happened:

- If **which pip** prints nothing, pip is not installed on that machine (or not on the PATH).

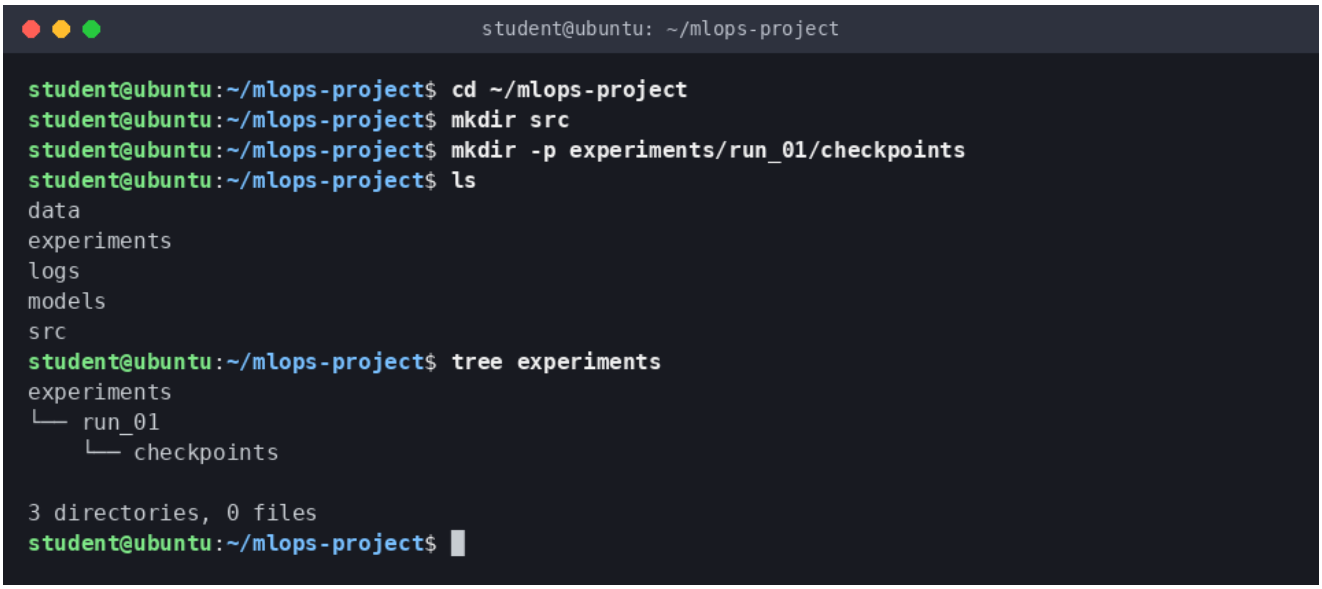
• Inside a virtual environment, **which python3** would show a path inside your project instead of /usr/bin.

Step 2: File and Directory Management

mkdir : Make Directory

Creates a new folder. Use **-p** to create nested folders in one go (and to avoid an error if it already exists). We will now build a small ML project folder that we use for the rest of the guide.

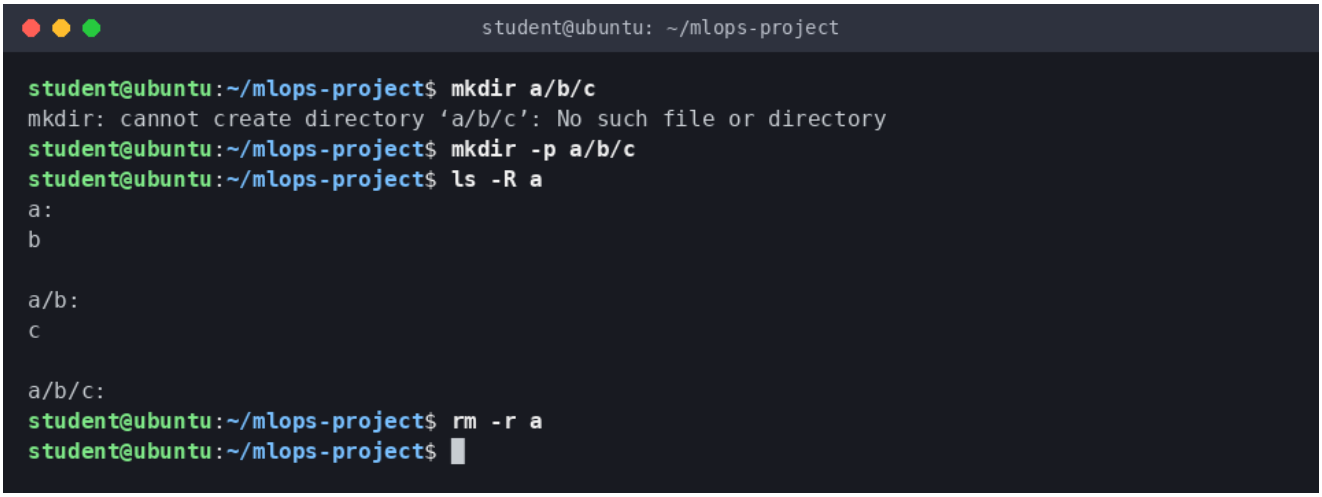
Screenshot 11: Creating folders. 'tree' draws the folder structure.



What happened:

- **mkdir src** makes one folder. **mkdir -p experiments/run_01/checkpoints** makes experiments, then run_01 inside it, then checkpoints inside that.
- **tree** is a handy picture of the structure. If it is missing, install it with **sudo apt install tree** or use **ls -R**.

Screenshot 12: Why -p matters.



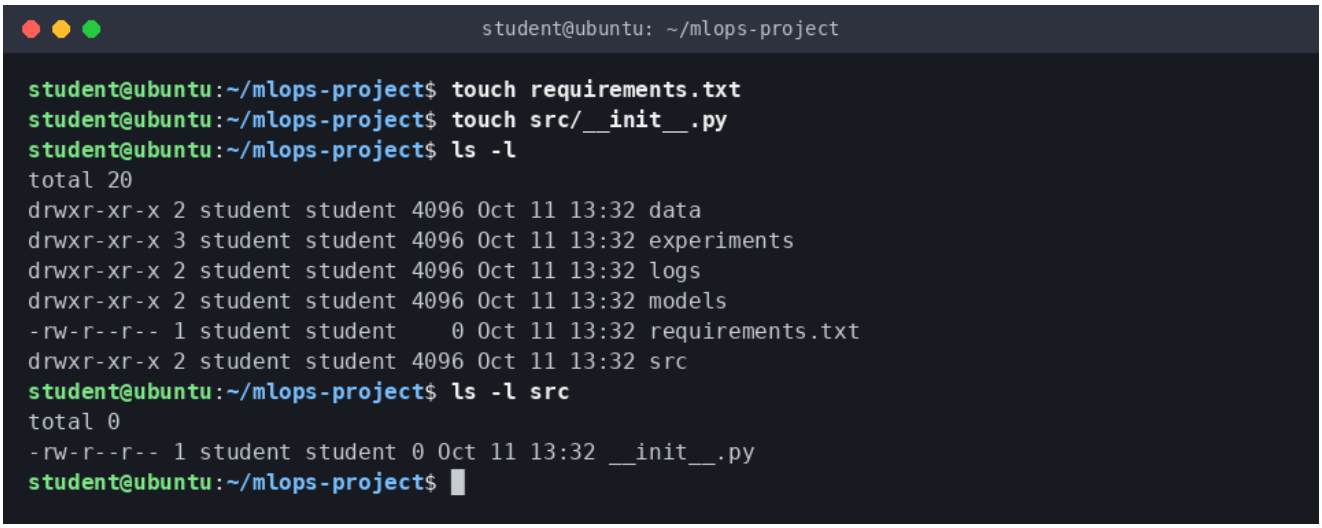
What happened:

- Without **-p**, mkdir refuses because the parent folder 'a/b' does not exist yet. With **-p** it creates the whole chain.
- We removed the demo folder with **rm -r** (explained below).

touch : create an empty file or update its timestamp

Creates an empty file if it does not exist (or refreshes its 'last modified' time if it does). Often used for placeholders like `__init__.py` or `.gitkeep`.

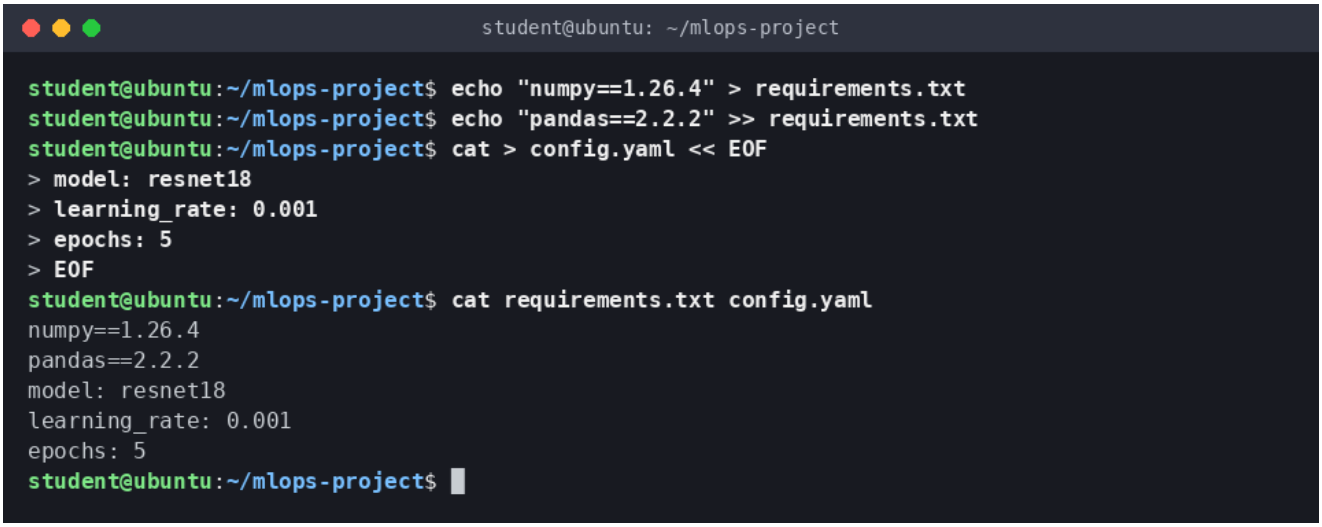
Screenshot 13: touch creates empty files (size 0).



Putting text into a file with echo and > (needed a lot in exams)

The sign **>** sends the output of a command into a file instead of the screen. One **>** *overwrites* the file; **>>** *appends* to the end. For several lines use a 'here-document': `cat > file << EOF` then your lines, then EOF on its own line.

Screenshot 14: Writing text to files without any editor.



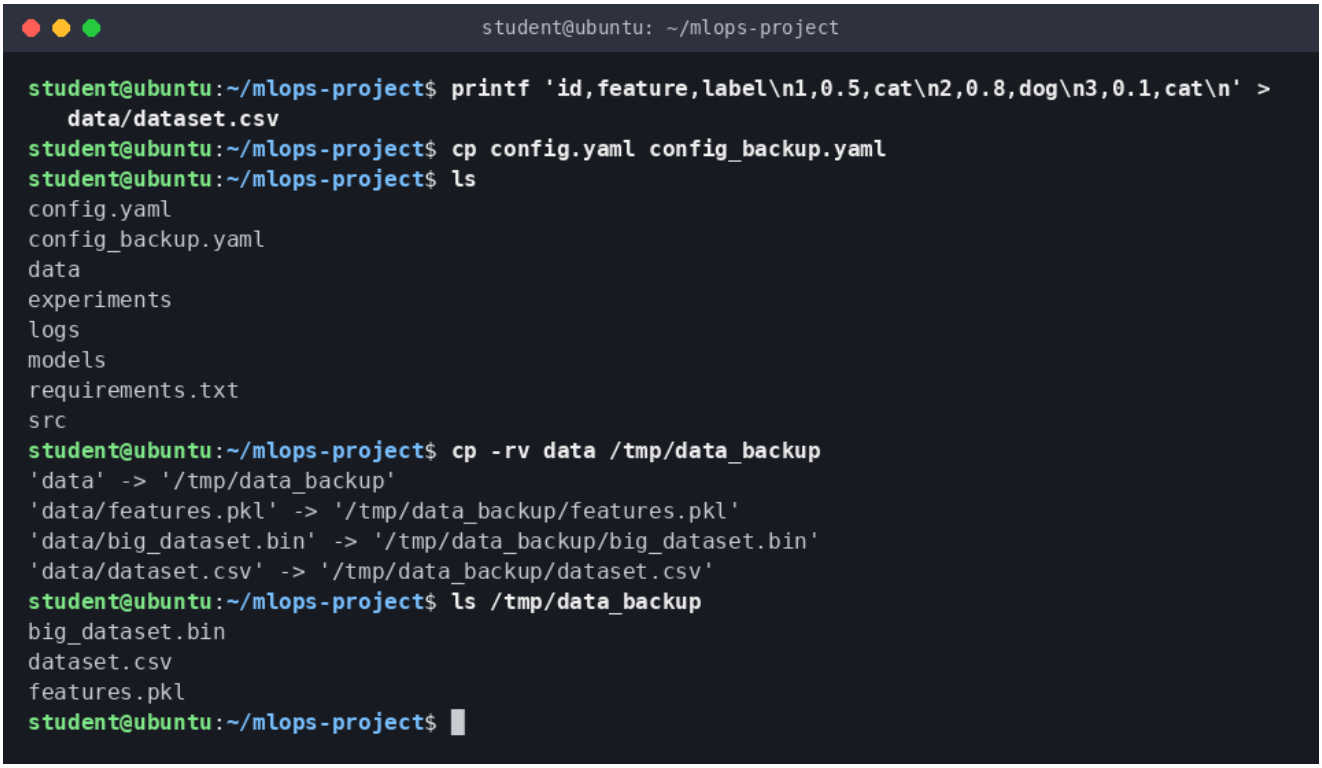
What happened:

- First **>** wrote one line, second **>>** added another line below it.
- The here-document wrote three lines into config.yaml. The prompt turned into **>** while waiting for more lines, until we typed EOF.
- **cat** displayed both files one after the other.

cp : Copy files or directories

cp source destination. Use **-r** (recursive) for folders and **-v** (verbose) to see what was copied.

Screenshot 15: Copying a file and a whole folder.



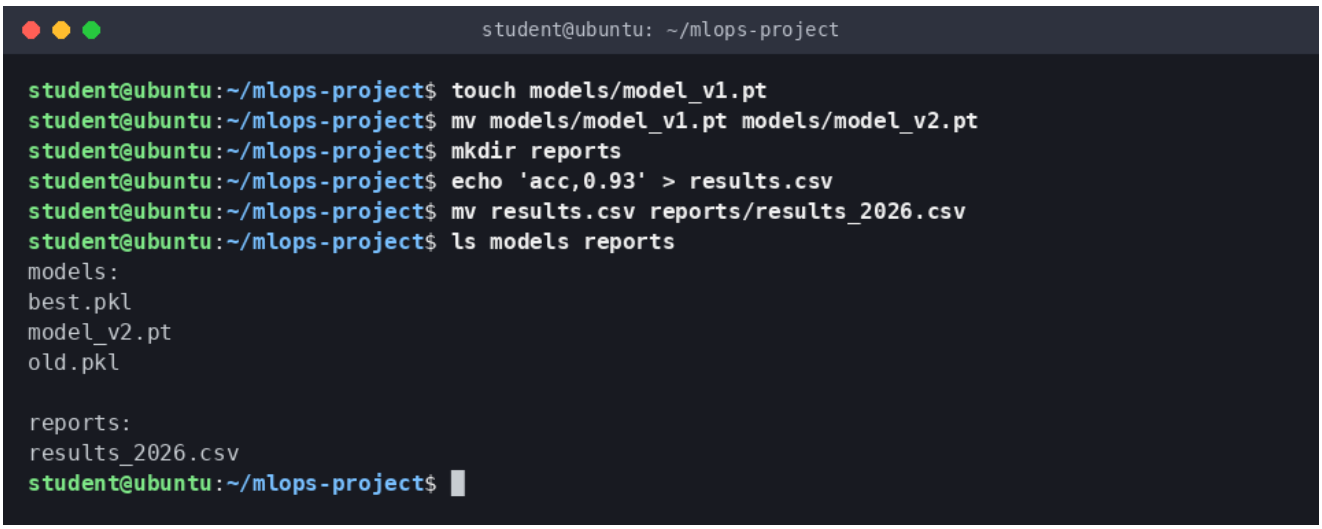
What happened:

- **cp config.yaml config_backup.yaml** makes a duplicate with a new name.
- Copying a folder needs **-r**. With **-v** it prints each item copied.

mv : Move or Rename

mv source destination. If the destination is in the same folder, it simply renames. Otherwise it moves.

Screenshot 16: Renaming a file and moving a file into another folder.

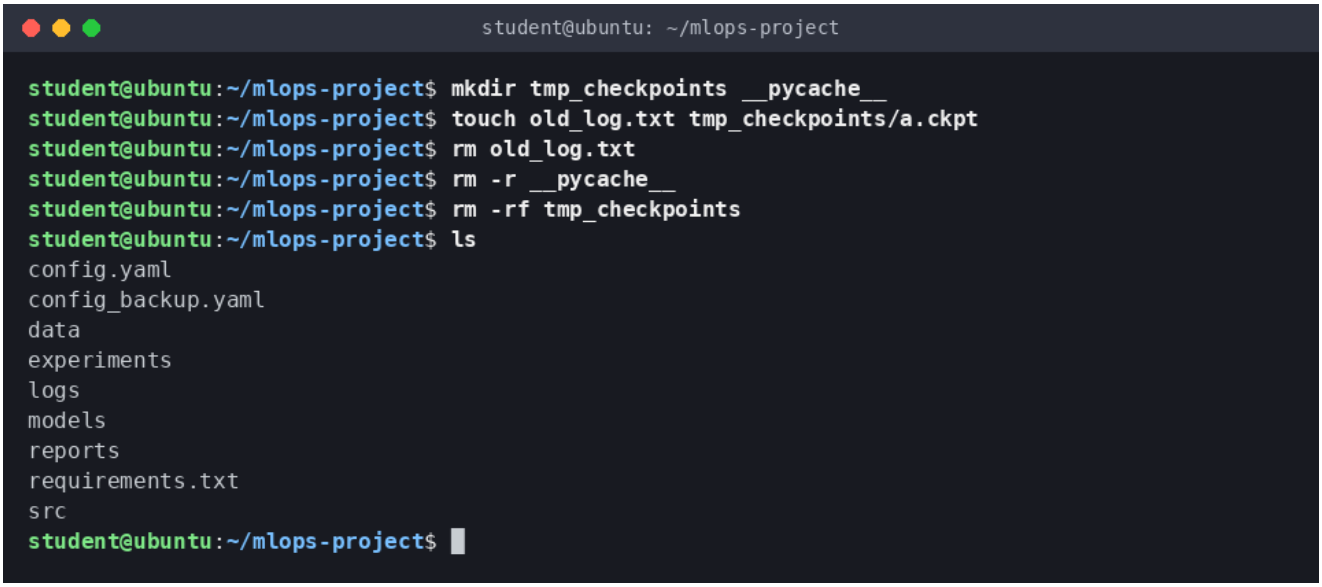


rm : Remove files and directories

Deletes files. **-r** deletes folders and everything inside; **-f** forces deletion without asking. **-i** asks before each deletion (safe for learning).

WARNING: rm is PERMANENT. There is no recycle bin. Never run **rm -rf** on a path you have not checked, and be extra careful with wildcards like * and with **rm -rf /**. Run **ls** on the same path first to see what will vanish.

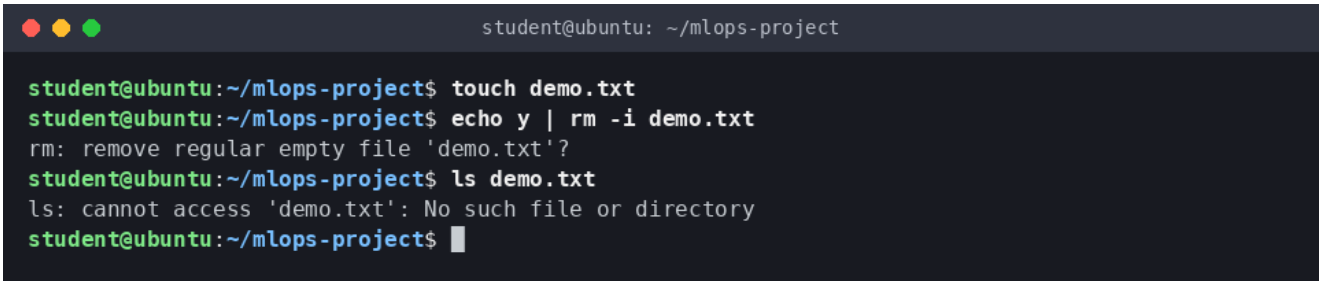
Screenshot 17: rm for a file, rm -r for a folder, rm -rf to force.



What happened:

- Deleting a normal folder with plain **rm** would give 'Is a directory'. That is why **-r** is needed.
- Afterwards **ls** proves they are gone.

Screenshot 18: Interactive delete: rm -i asks 'remove?' first (we answered y).



What happened:

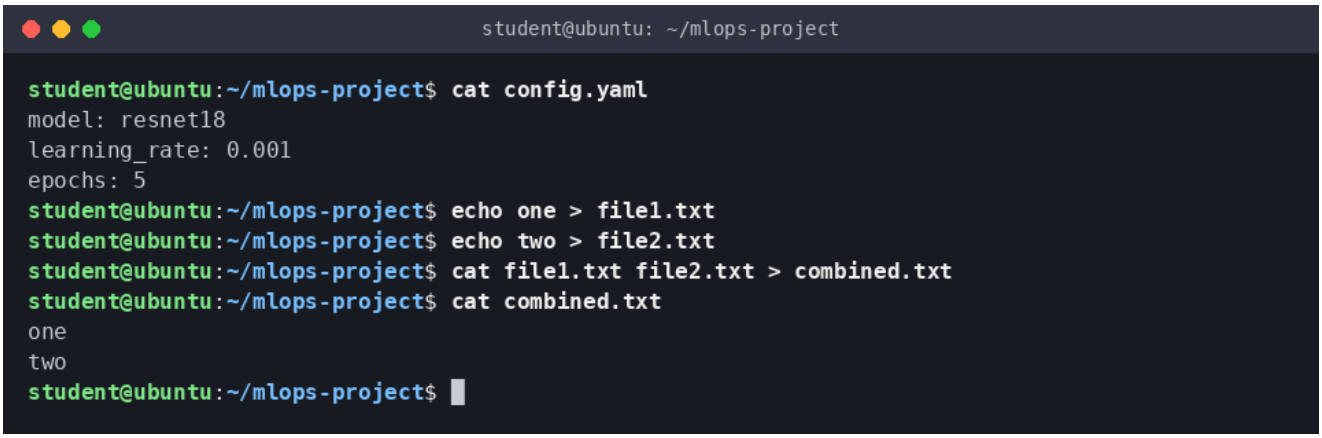
- Without the 'echo y |' part you would type y yourself and press Enter. The last ls confirms the file is gone.

Step 3: Viewing and Editing File Contents

cat : show a whole file

Prints the full file on the screen. Good for short files. It can also join several files: `cat file1 file2 > combined.txt`.

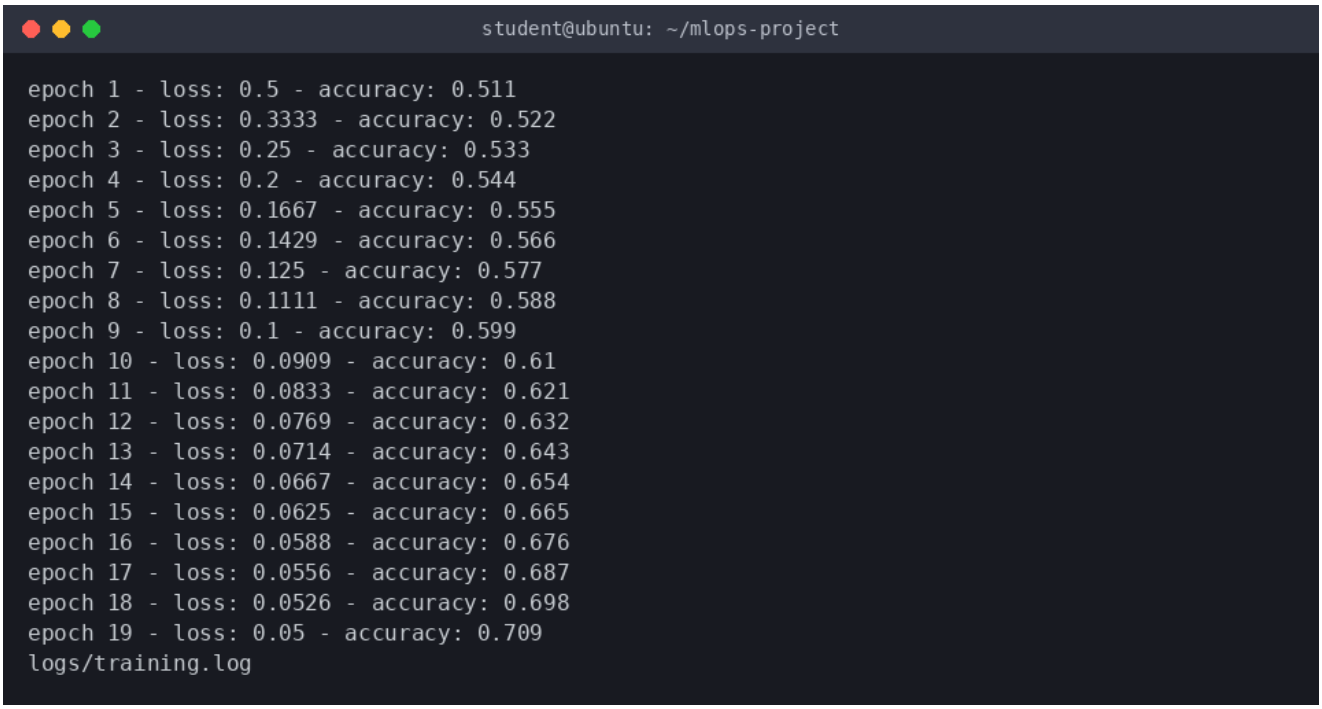
Screenshot 19: cat shows files and can merge them.



less (and more) : page through a big file

Shows a file one screen at a time. Keys: **Space** = next page, **b** = back, arrow keys scroll, **/word** searches, **n** = next match, **q** = quit. Better than cat for big logs.

Screenshot 20: Inside less (this is a real screen capture). Press q to leave.



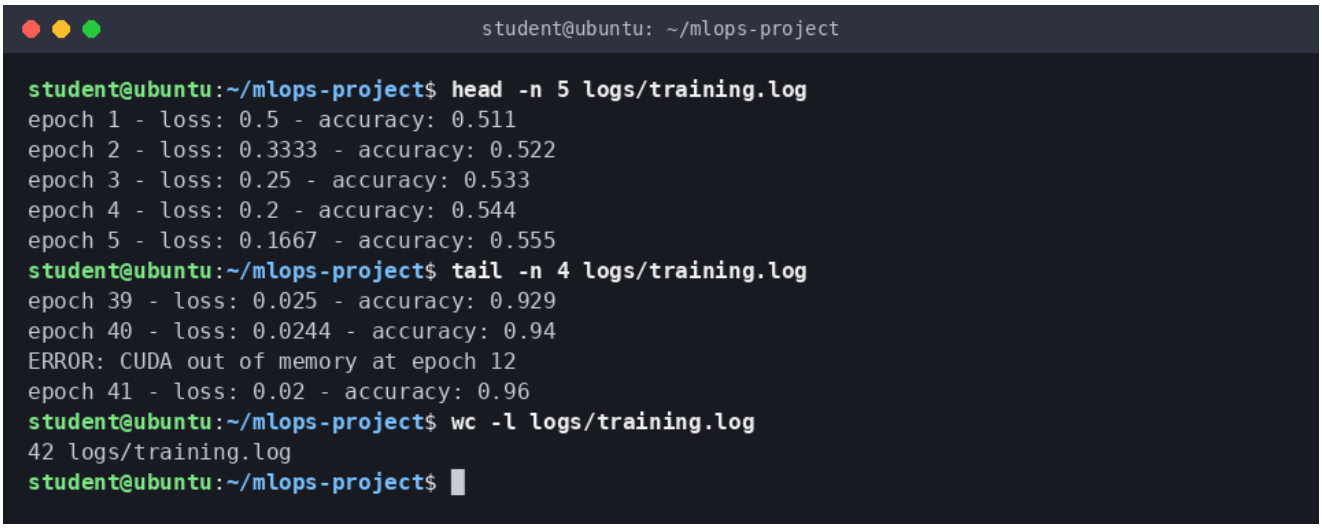
What happened:

- The last line (a lone colon) is where less waits for your key. Press Space for the next page. **Remember q to quit**, otherwise it looks like the terminal is frozen.

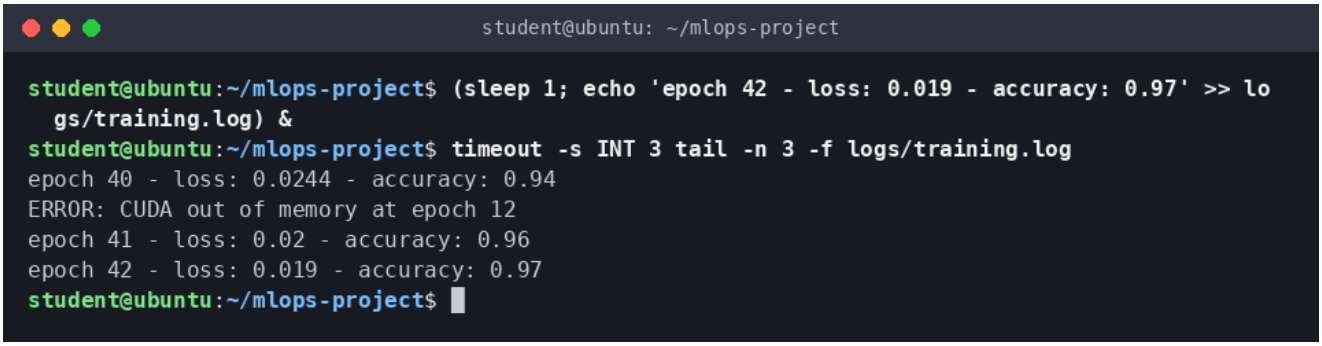
head and tail : first or last lines

head shows the start (10 lines by default), **tail** shows the end. **-n 5** chooses how many. **tail -f** keeps the file open and shows new lines as they are written: perfect for watching a training log live.

Screenshot 21: First 5 lines, last 4 lines, and the line count.



Screenshot 22: tail -f following a log. A new line appeared while it was watching.



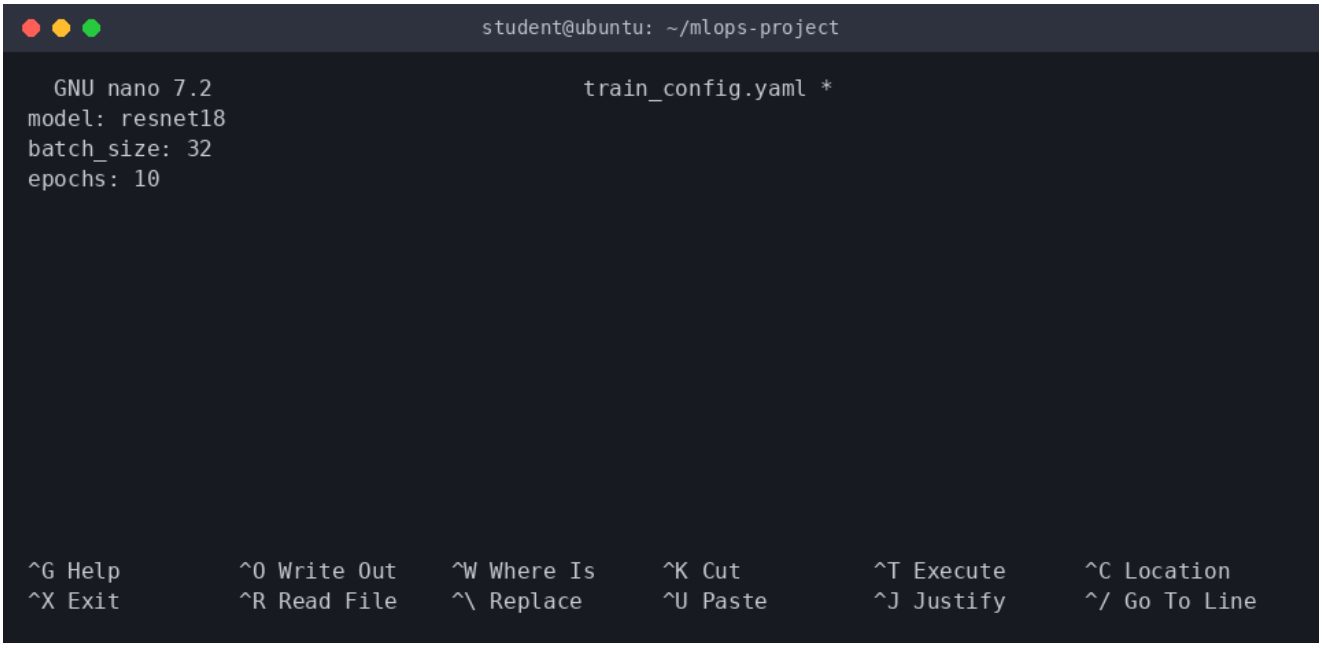
What happened:

- We simulated a training job writing a new line after 1 second. In real life you run **tail -f logs/training.log** and press **Ctrl + C** when you want to stop watching (here **timeout 3** stopped it automatically after 3 seconds so the screenshot could be taken).

nano and vim : edit files in the terminal

On servers there is no mouse or notepad, so you edit text inside the terminal. **nano** is easiest because the shortcuts are shown at the bottom (^ means Ctrl). **vim** is more powerful but trickier.

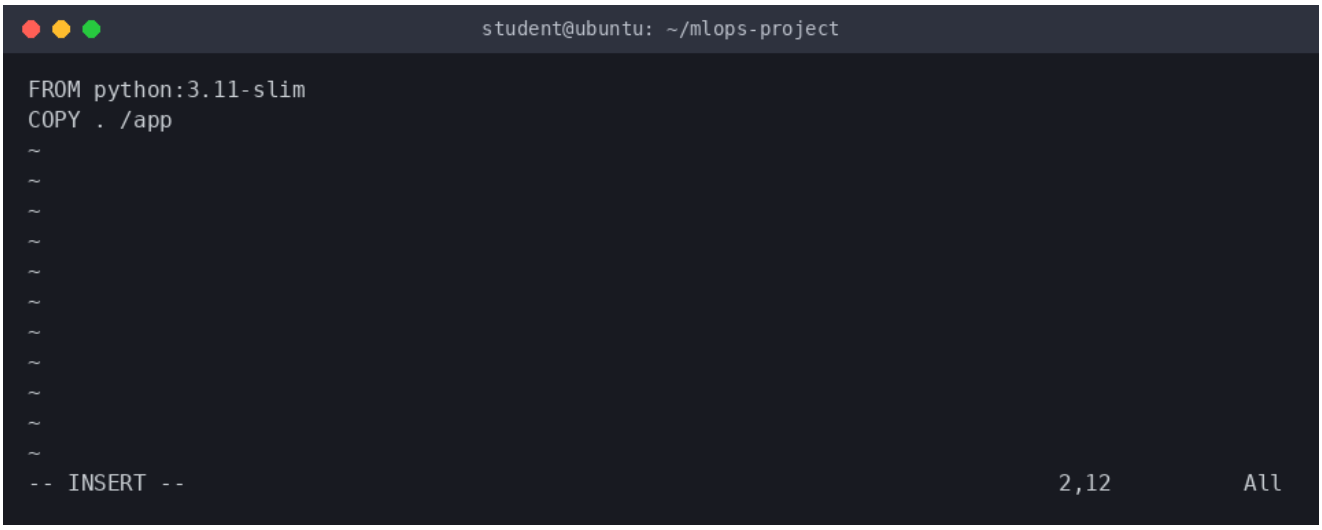
Screenshot 23: nano with some text typed in (real capture).



What happened:

- Type your text normally. Save with **Ctrl + O** then **Enter**. Exit with **Ctrl + X**. If you forget to save, nano asks 'Save modified buffer?' Press **Y**.

Screenshot 24: vim in INSERT mode (real capture).



What happened:

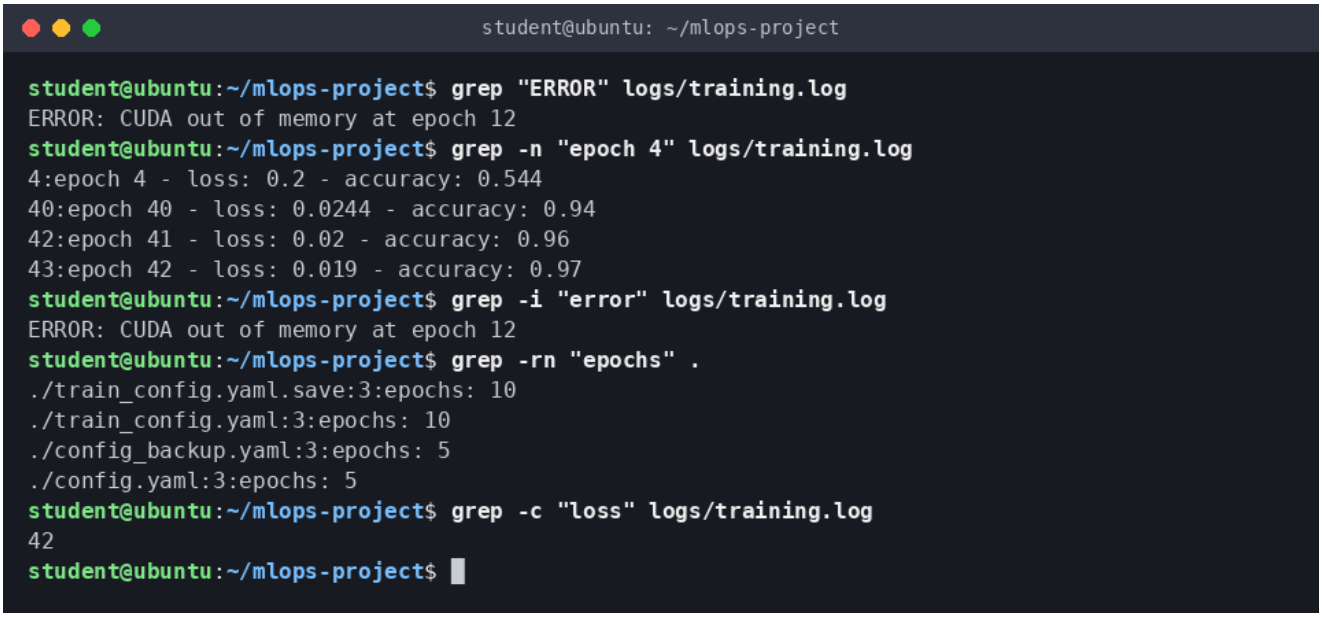
- vim starts in **normal mode** where keys are commands. Press **i** to enter **INSERT mode** (you see '-- INSERT --' at the bottom) and then type.
- To save and quit: press **Esc**, then type **:wq** and press Enter. To quit WITHOUT saving: **Esc** then **:q!** and Enter.

> **EXAM HINT:** Stuck in vim? Press Esc, then type :q! and Enter. If an exam allows it, use nano; it is far less confusing.

grep : search text inside files

Prints only the lines that match a pattern. Flags: **-i** ignore upper/lower case, **-r** search all files in a folder, **-n** show line numbers, **-c** count matches.

Screenshot 25: grep in action.



What happened:

- **grep -n** adds the line number at the start of each match.
- **grep -rn "epochs" .** searched every file in the current folder (and below) and told us the file name and line number.

Step 4: File Permissions and Ownership

Every file has an owner and a set of permissions that say who may **r**ead, **w**rite or e**x**ecute it. Look at the start of an `ls -l` line, for example `-rwxr-xr--`:

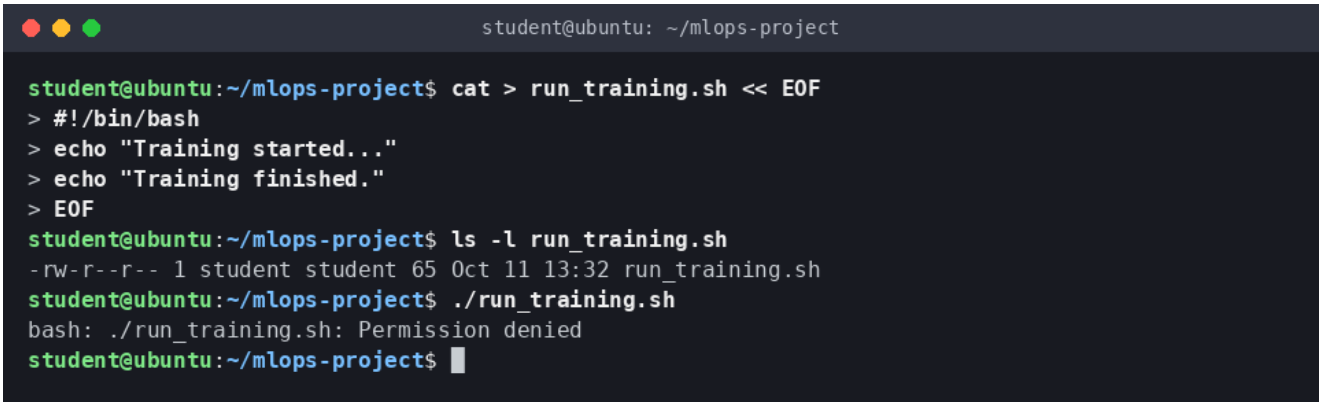
Part	Example	Meaning
1st char	-	file type: - file, d directory
next 3	rwx	owner (user) permissions
next 3	r-x	group permissions
last 3	r--	everyone else (others)

Number	Meaning	Number	Meaning
r = 4	read	7 = 4+2+1	rwx (all)
w = 2	write	6 = 4+2	rw-
x = 1	execute	5 = 4+1	r-x
		4	r--

chmod : change permissions

Set permissions with numbers (**755**, **644**) or letters (**u+x** = add execute for the user). A shell script must have execute permission before you can run it with **./name**.

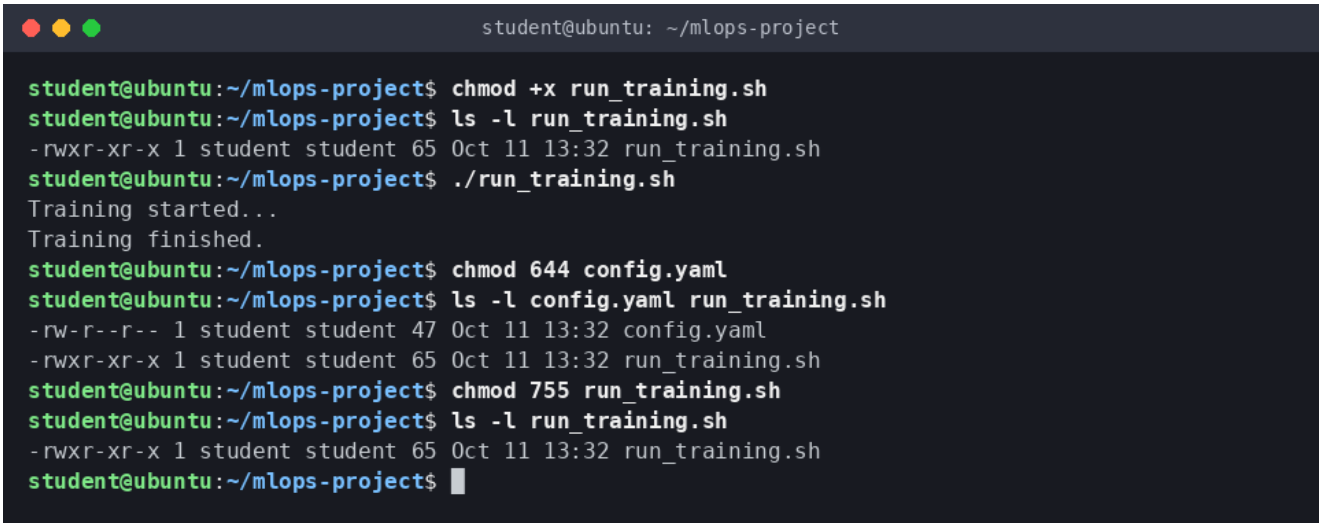
Screenshot 26: A new script cannot be run yet: 'Permission denied'.



What happened:

- **-rw-r--r--** has no x anywhere, so the script is not executable.
- **#!/bin/bash** on the first line tells Linux which program runs the script.

Screenshot 27: After chmod +x the script runs. 644 and 755 explained.



What happened:

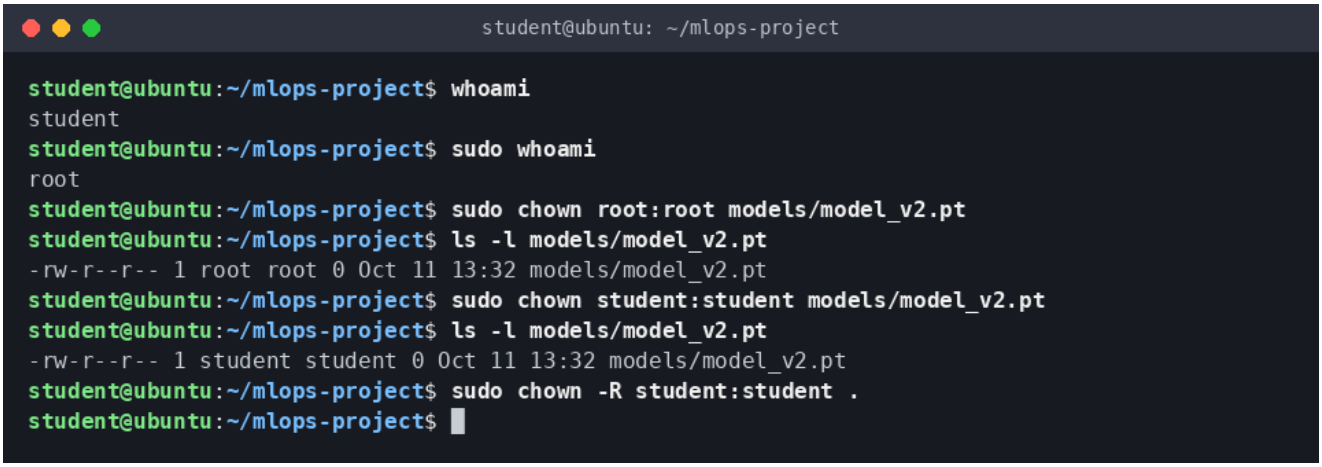
- Now we see **-rwxr-xr-x**. The x flags appeared.
- **644** = owner can read+write, others only read (normal for config files). **755** = owner can do everything, others read+execute (normal for scripts).
- **./** before the name means 'run the file in this folder'.

chown : change owner

Changes the user (and optionally group) that owns a file. Needs administrator rights, so it is used with **sudo**. Typical case: files created inside a Docker container end up owned by root on the host and you must take them back.

sudo : run one command as administrator

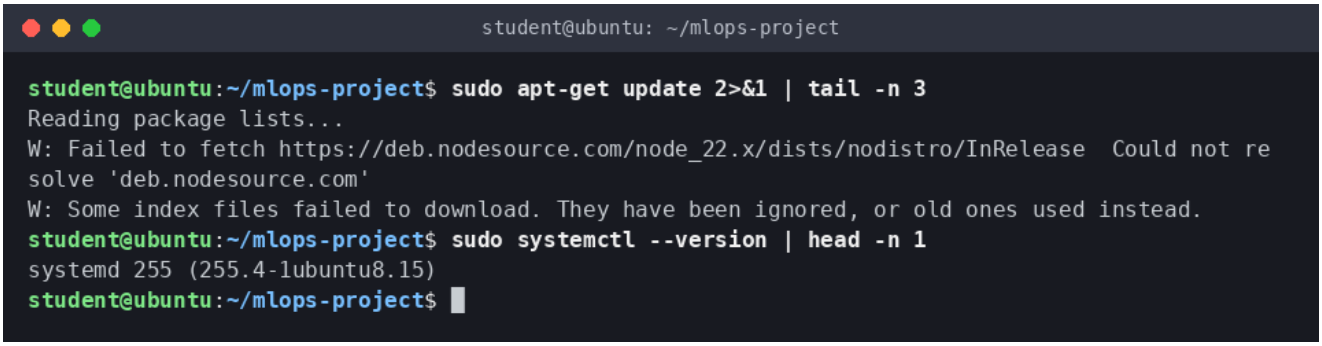
Screenshot 28: sudo and chown.



What happened:

- **sudo whoami** printed **root**, proof that sudo runs the command as the administrator.
- We gave the file to root, saw the owner change in **ls -l**, then took it back. **-R** applies to a folder and everything in it.
- On a real machine sudo asks for your password the first time.

Screenshot 29: sudo for system tasks (updating package lists, managing services).



What happened:

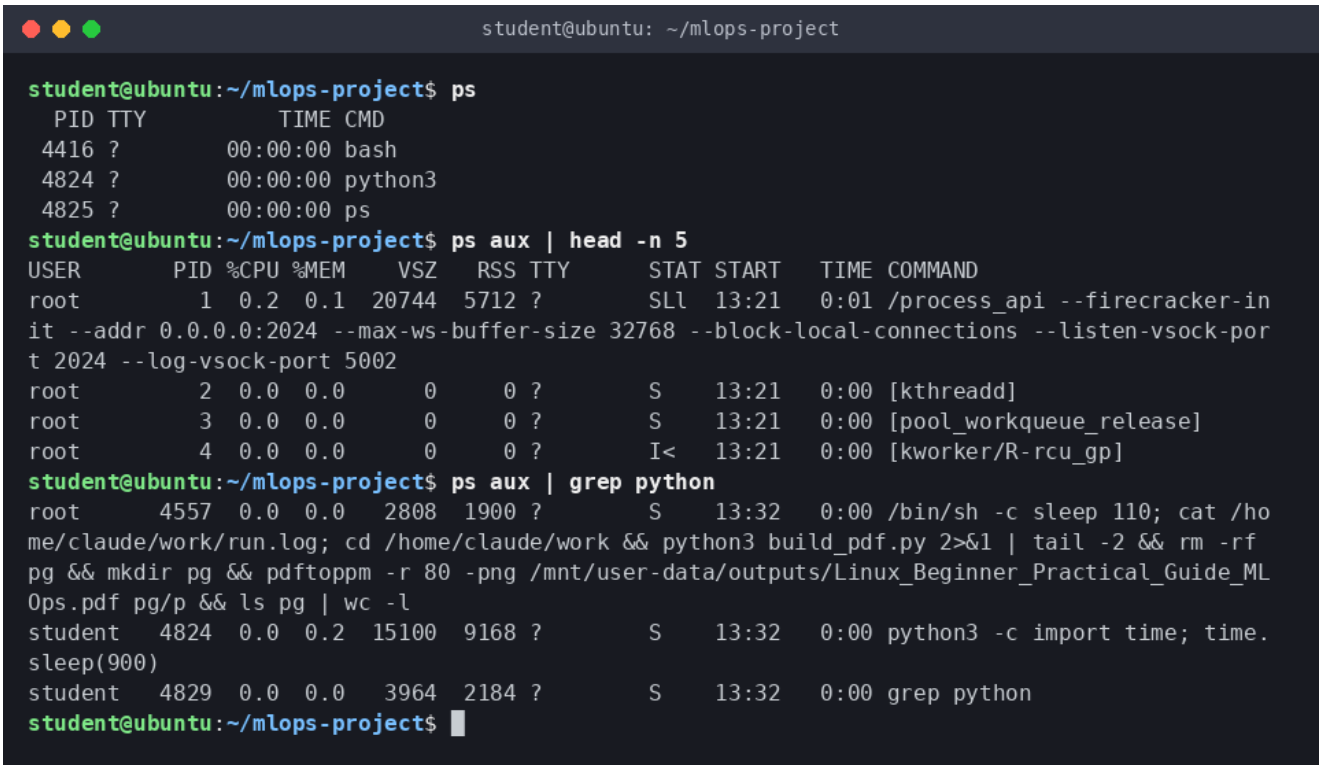
- **apt** is the package manager of Ubuntu: **sudo apt install htop** installs a program. **sudo systemctl restart docker** restarts the Docker service.

Step 5: Process and Job Management

A **process** is a running program. Each has a number called the **PID** (process ID). These commands let you find processes, watch them and stop them.

ps : list processes

Screenshot 30: ps shows processes. 'ps aux | grep python' filters to Python jobs.



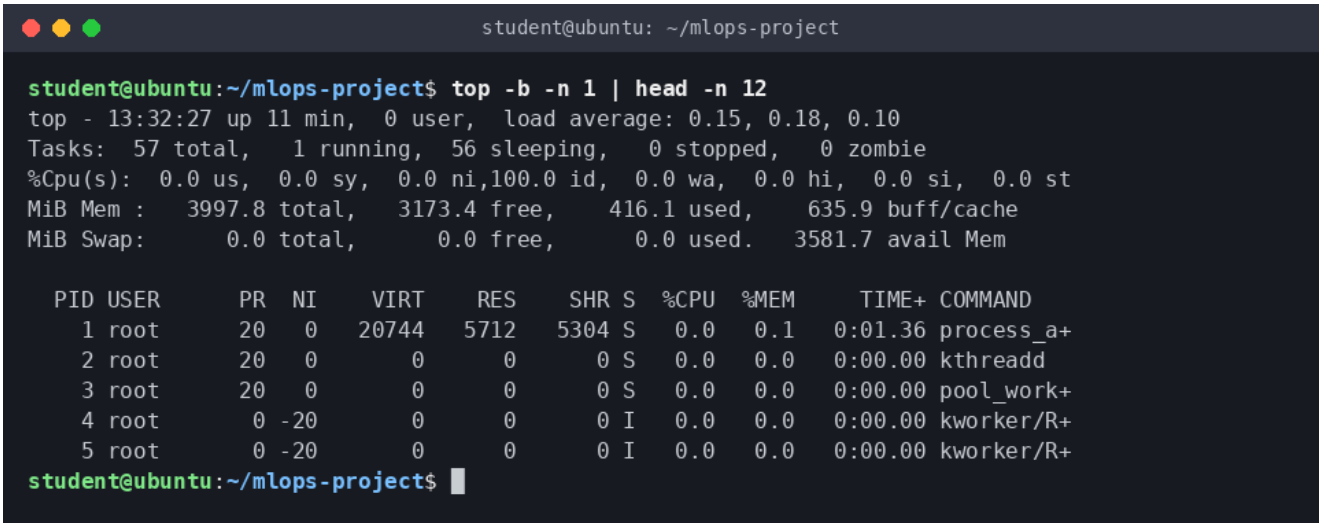
What happened:

- Plain **ps** shows only your current terminal. **ps aux** shows every process with user, PID, CPU% and memory%.
- The pipe **|** passes the long list to **grep python**, which keeps only lines containing 'python'. Find the PID in the second column. (The last line is the grep command itself, ignore it.)

top and htop : live monitor

top updates every few seconds and shows CPU, memory and the busiest processes. **htop** is a nicer coloured version. Press **q** to quit. Below, **top -b -n 1** prints one snapshot (b = batch mode) so it fits in a screenshot.

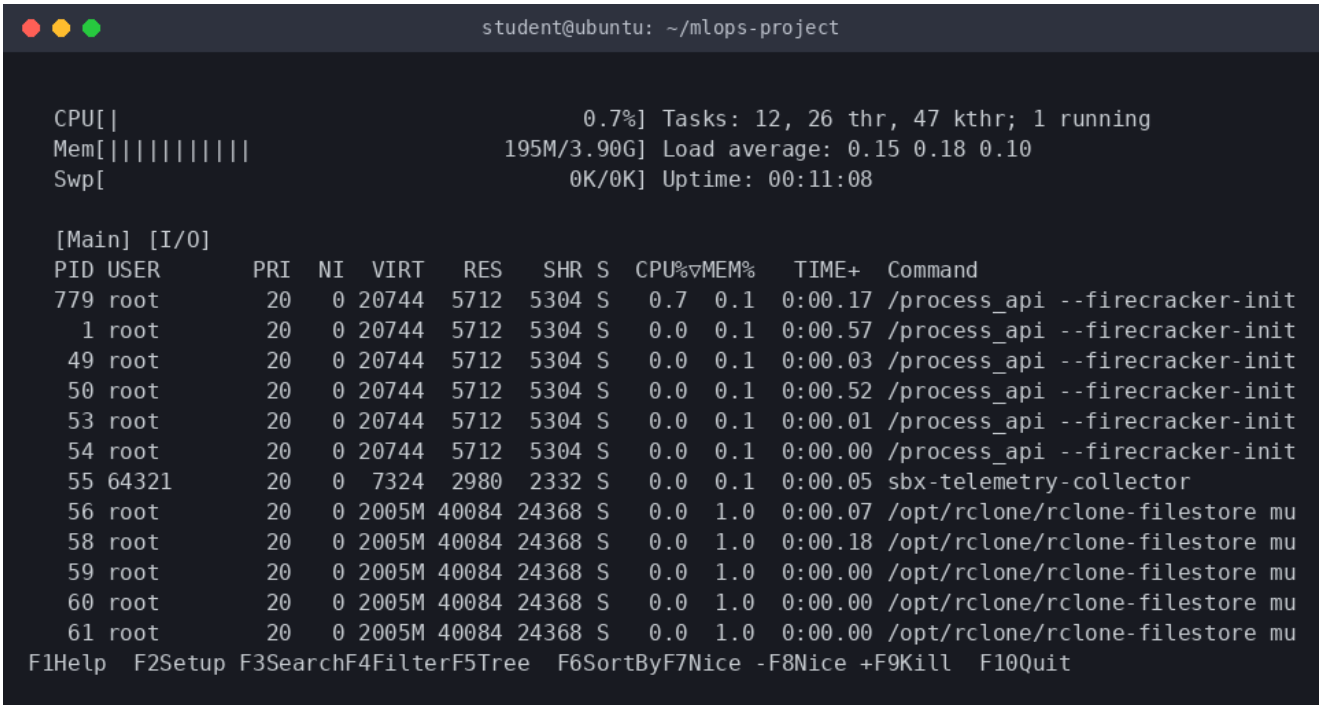
Screenshot 31: A snapshot of top.



What happened:

- Top lines: uptime and load. Then CPU and memory totals. Then a table of processes sorted by CPU use.

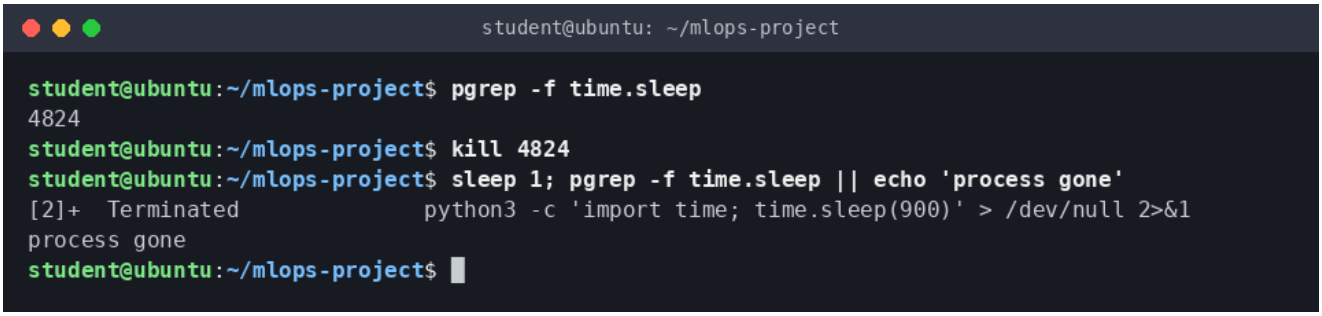
Screenshot 32: htop (real capture). Press q to quit.



kill and kill -9 : stop a process

kill PID politely asks the process to stop (graceful). If it ignores you, **kill -9 PID** forces it to die immediately (it gets no time to save anything). Always try plain kill first.

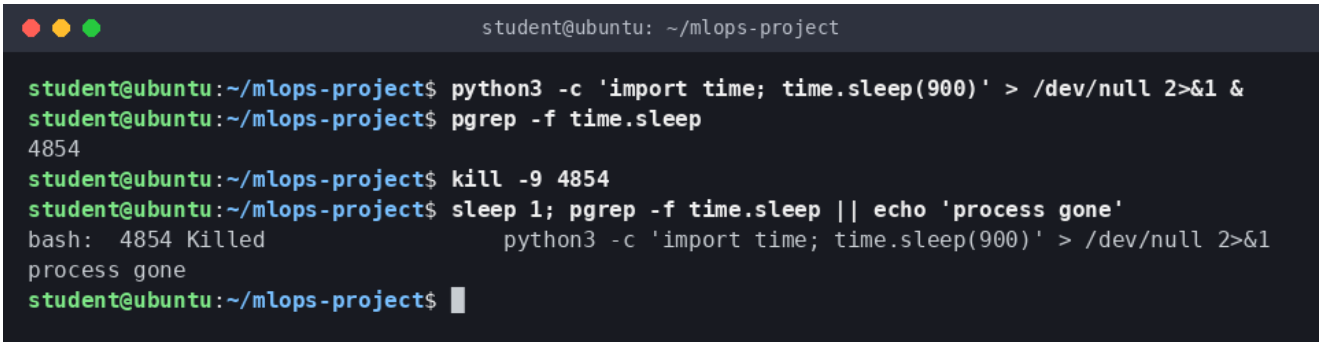
Screenshot 33: Finding a PID and killing it.



What happened:

- **pgrep -f** prints the PID of a process that matches the name. We used that number in **kill**.
- Afterwards pgrep finds nothing, so the process is gone.

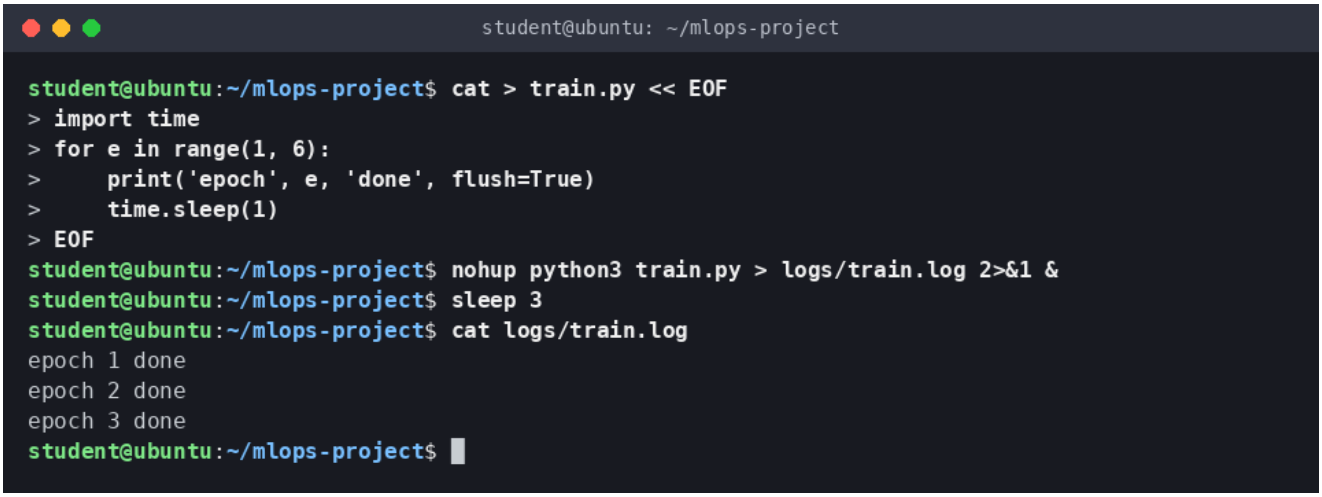
Screenshot 34: A forced kill with kill -9.



& and nohup : run jobs in the background

Adding **&** at the end runs the command in the background so you can keep typing. **nohup** additionally keeps it running after you close the terminal or lose your SSH connection, and saves its output in a file.

Screenshot 35: Starting a training script with nohup and reading its log.



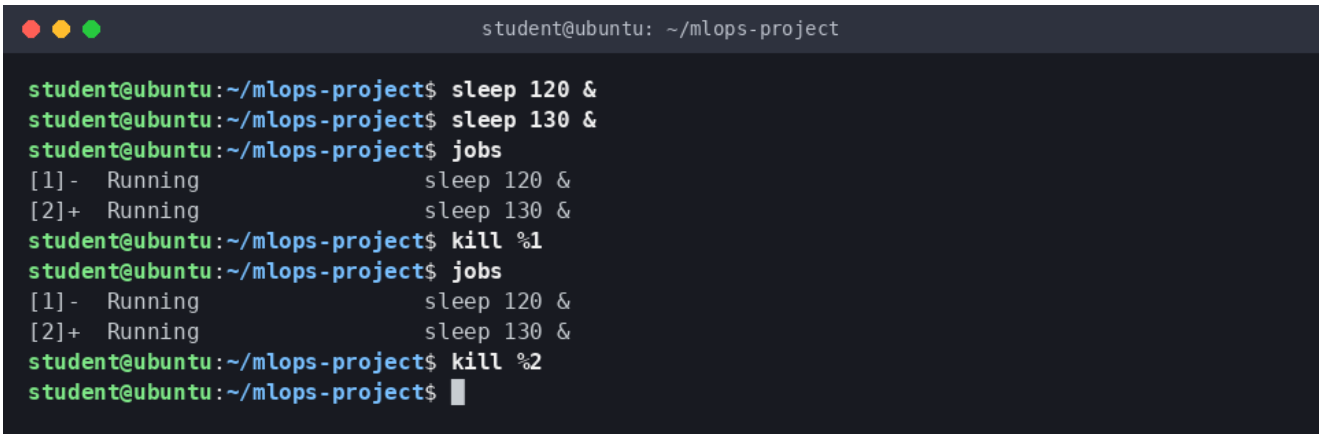
What happened:

- **> logs/train.log** sends normal output to the file. **2>&1** sends error messages to the same file. The final **&** backgrounds the job.
- After 3 seconds the log already shows progress while the terminal stayed free.

jobs, fg, bg : manage background jobs

jobs lists background jobs of this terminal. **fg %1** brings job 1 to the foreground. **Ctrl + Z** pauses the foreground job, and **bg** resumes it in the background.

Screenshot 36: jobs lists background jobs; kill %1 stops job number 1.



What happened:

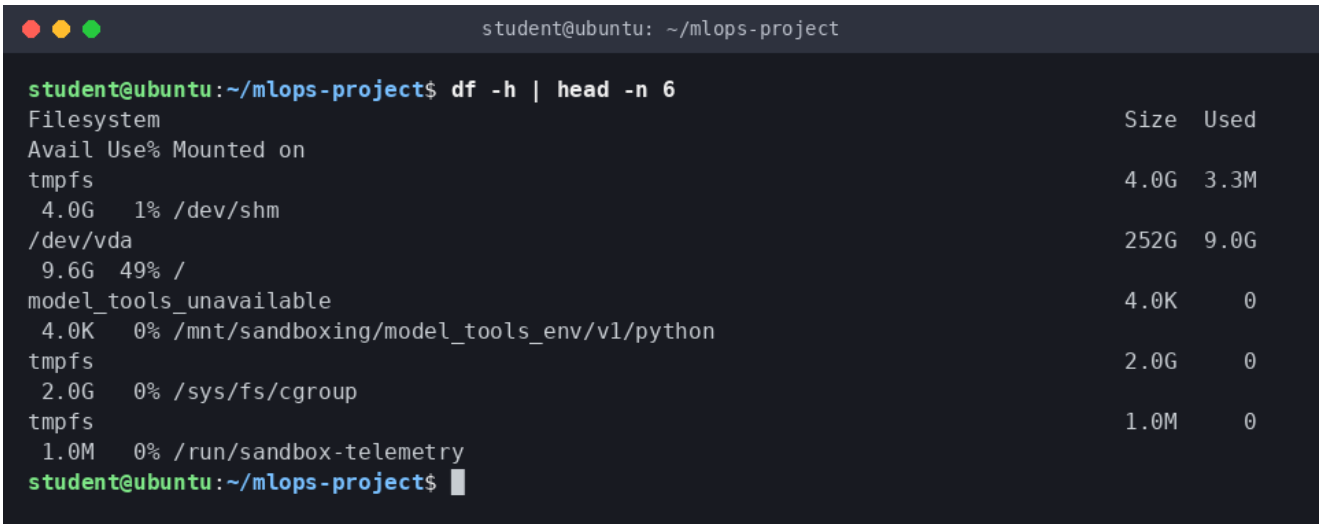
- **[1]** and **[2]** are job numbers (not PIDs). The + marks the most recent job.
- Use **%** before the job number: **kill %1**, **fg %1**.

Step 6: System Monitoring and Resource Usage

df : free disk space

Shows space used and available on each disk. **-h** gives human-readable sizes. Check this before downloading a dataset or saving a checkpoint.

Screenshot 37: df -h.



What happened:

- **Size** = total, **Used**, **Avail** = free, **Use%** = how full, **Mounted on** = where the disk appears.

du : how big is this folder?

du -sh folder = total size of a folder (s = summary). Combine with **sort** to find the biggest folders.

Screenshot 38: Disk usage of a folder and sorting by size.

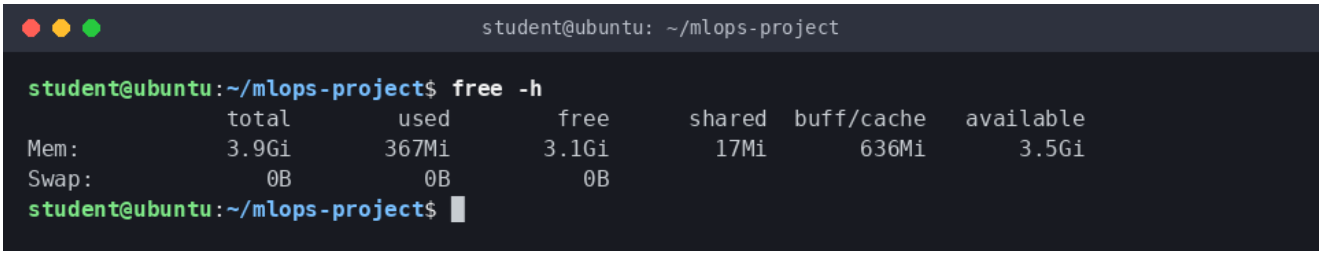


What happened:

- The biggest items appear at the bottom with **sort -h**. The 200 MB dummy dataset we created is the culprit.

free : memory (RAM) usage

Screenshot 39: free -h.



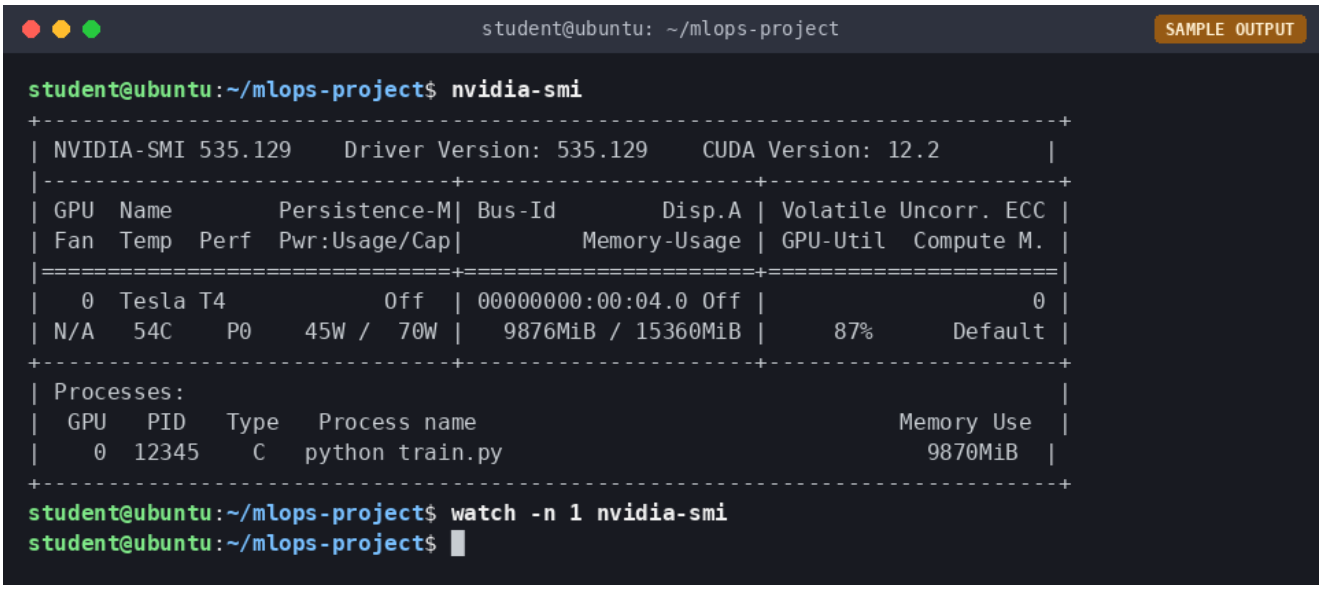
What happened:

- Look at the **available** column for memory that programs can still use. 'Out of memory' errors during training mean this is near zero.

nvidia-smi : GPU status

Shows GPU usage, memory, temperature and which processes use the GPU. It exists only on machines with an NVIDIA GPU and driver. The Linux machine used for this guide has no GPU, so the screenshot below is a **typical sample**.

Screenshot 40: Sample nvidia-smi output on a GPU machine.



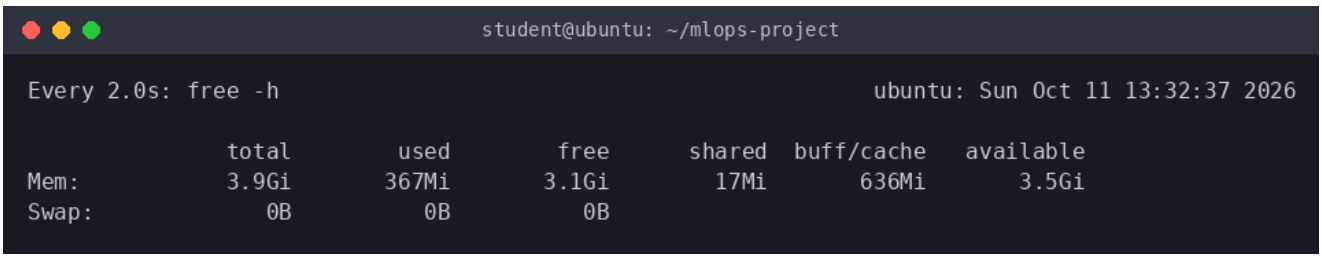
What happened:

- Read: **GPU-Util 87%** = the GPU is busy. **Memory-Usage 9876MiB / 15360MiB** = VRAM used out of total. The Processes table shows **python train.py** is the user.
- **watch -n 1 nvidia-smi** repeats it every second. Press Ctrl + C to stop.

watch : repeat a command automatically

watch -n 5 df -h re-runs `df -h` every 5 seconds and shows the fresh result. Quit with Ctrl + C.

Screenshot 41: watch re-running free -h (real capture).

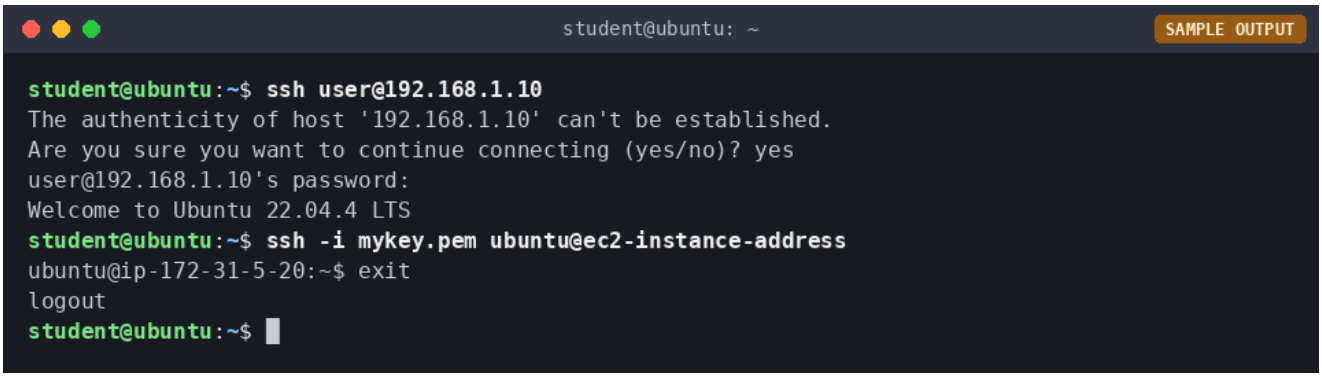


Step 7: Networking and Remote Access

ssh : log in to another computer

ssh user@address opens a terminal on a remote server (training machine, cloud VM). It needs the server's address and a password or a key file. A real server is needed to try this, so the screenshot is a **sample**.

Screenshot 42: Sample ssh sessions.



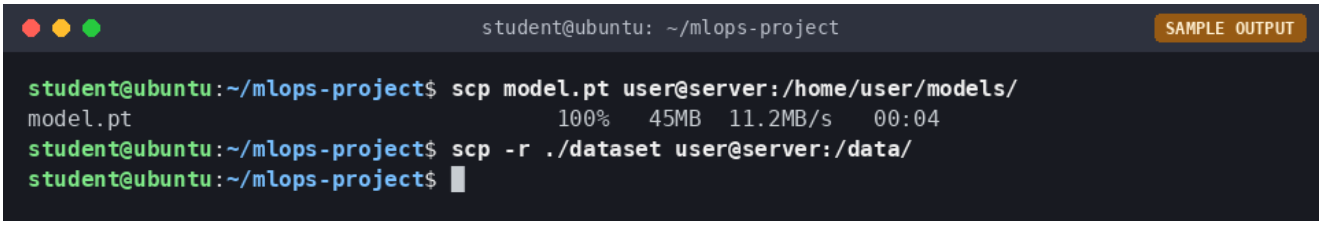
What happened:

- First connection asks you to type **yes**. The password you type is invisible. For cloud machines you give the key file with **-i**. Type **exit** to come back.
- If you get 'Permission denied (publickey)' on a .pem key, run **chmod 400 mykey.pem** first.

scp : copy files over ssh

scp source destination. Remote paths look like `user@server:/folder/`. Use **-r** for folders.

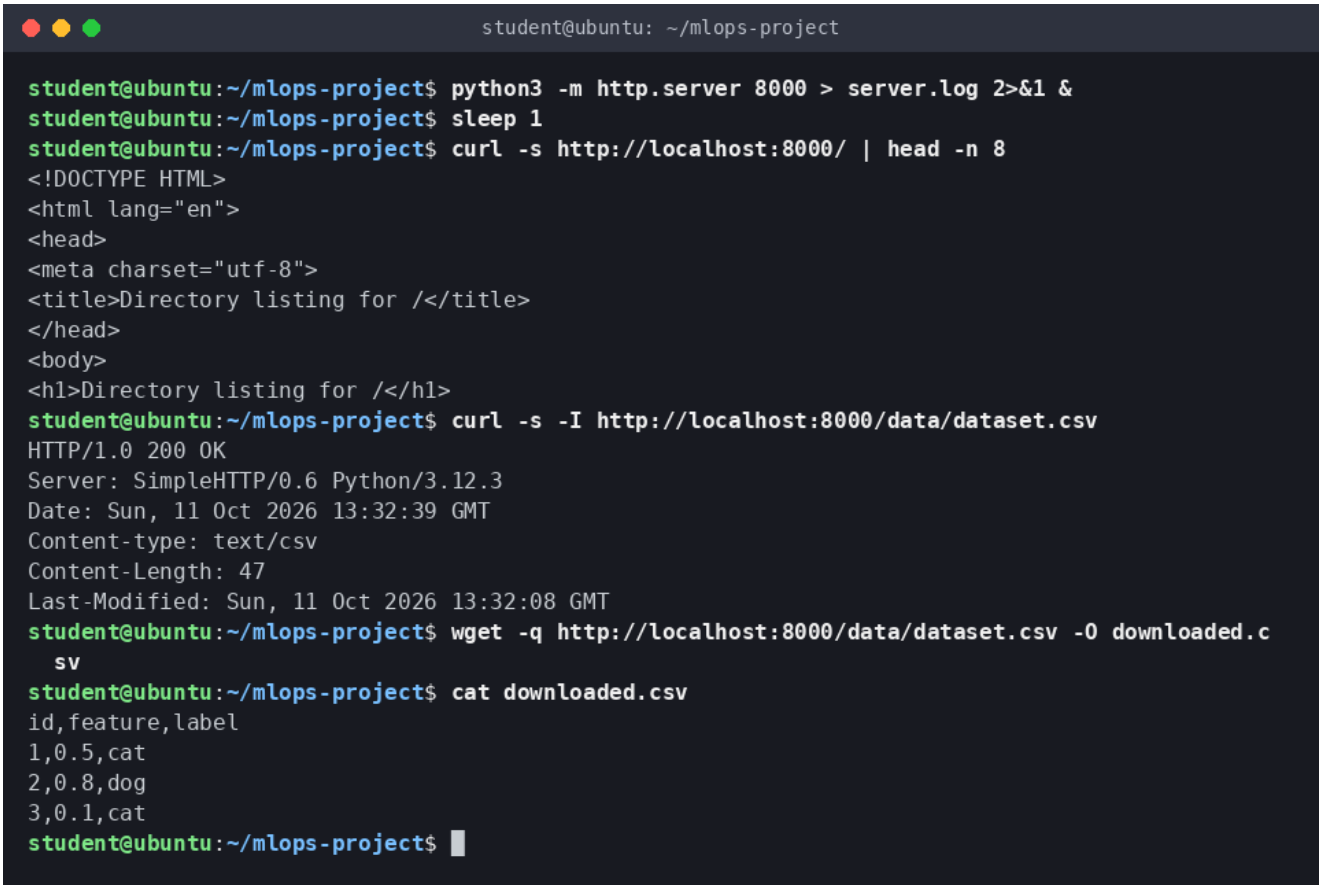
Screenshot 43: Sample scp output.



curl and wget : download files or call an API

wget URL downloads a file. **curl URL** prints what the web address returns (great for testing a deployed model API). We start a tiny web server on this very machine so both commands can be tried for real. **localhost** means 'this computer'.

Screenshot 44: A local web server, curl and wget.

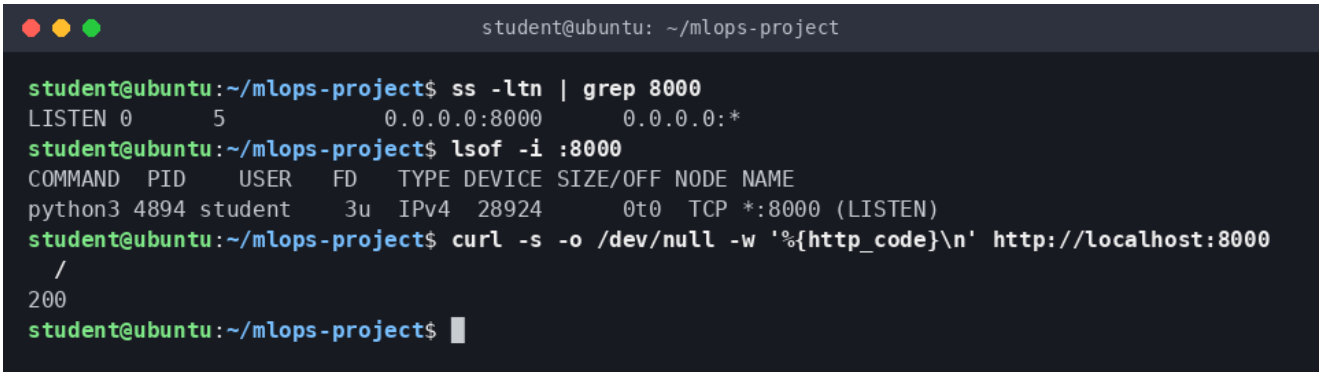


What happened:

- **curl -s** = silent (no progress bar). **curl -I** shows only the response header; **200 OK** means success.
- **wget -O name** saves under the name you choose. For an ML model API you would write: **curl -X POST http://localhost:8000/predict -d '{"x":1}'**

Checking which program is using a port

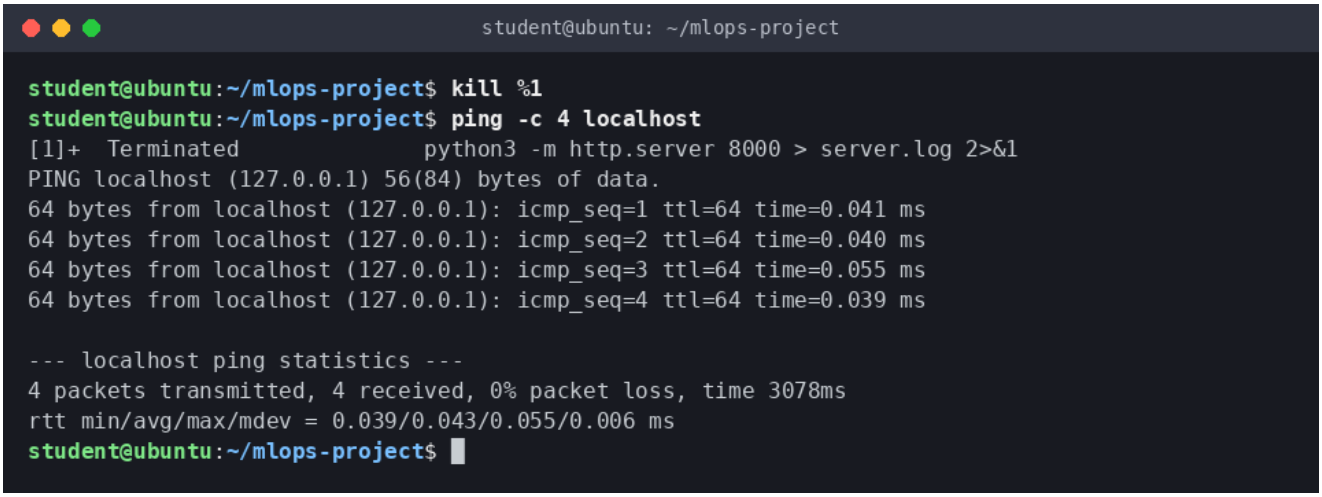
Screenshot 45: Is something listening on port 8000?



What happened:

- **LISTEN** on port 8000 means the server is up. **lsof -i :8000** tells which program (python3) and PID own the port.
- The last command prints just the HTTP status code (200). Same idea works for a model server's **/health** URL.

Screenshot 46: Stopping the server and testing connectivity with ping.



What happened:

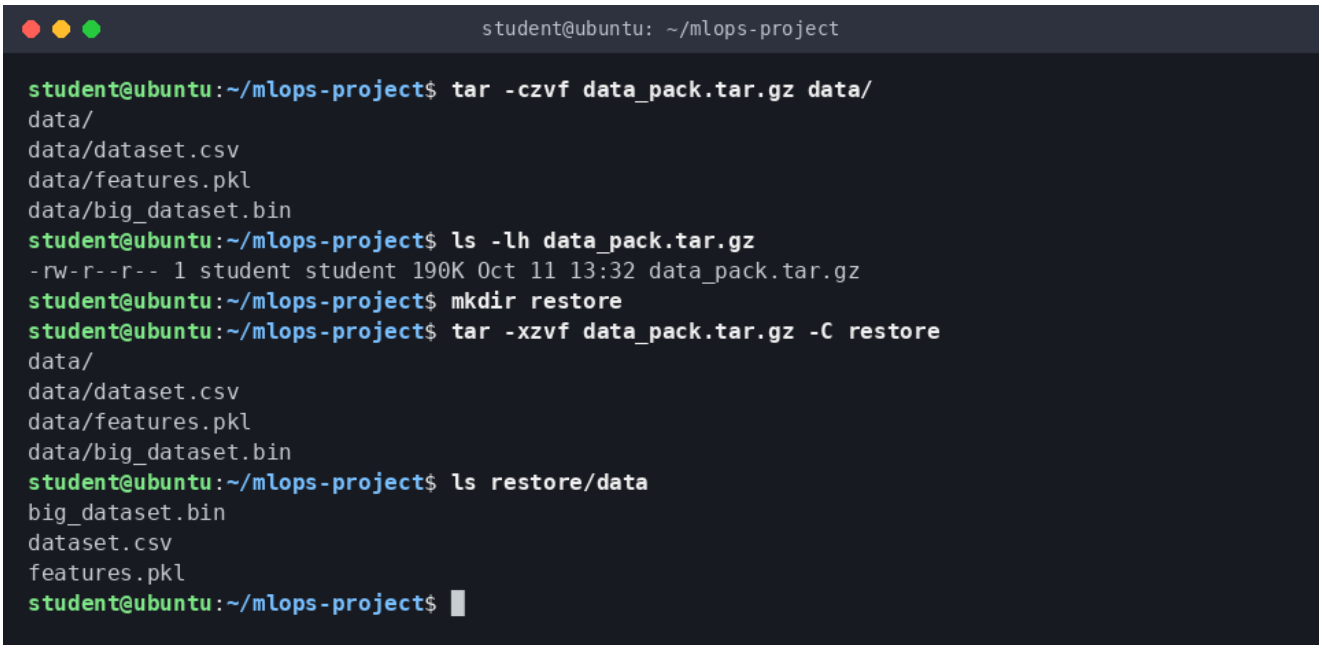
- **ping -c 4 host** sends 4 test packets. **time=** is the latency in milliseconds. 'Destination Host Unreachable' or 100% loss means a network problem. (Use a real address like 8.8.8.8 on an internet-connected machine.)

Step 8: Archiving, Compression and Transfer

tar : bundle files into one archive

Flags: **-c** create, **-x** extract, **-z** gzip compression, **-v** verbose, **-f** file name (must be last, right before the name). Memory trick: create = **czvf**, extract = **xzvf**.

Screenshot 47: Create and extract a tar.gz archive.

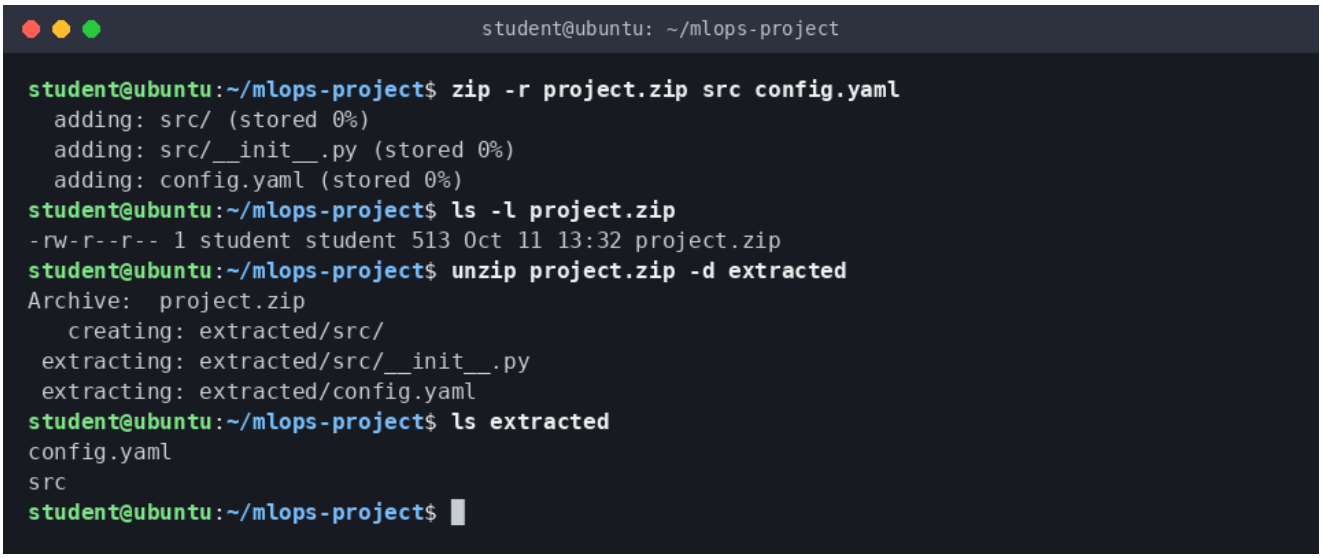


What happened:

- **-C restore** extracts into the folder 'restore'. The big 200 MB file compressed to a tiny size because it is full of zeros; real data compresses less.

zip and unzip

Screenshot 48: Creating and extracting .zip files.



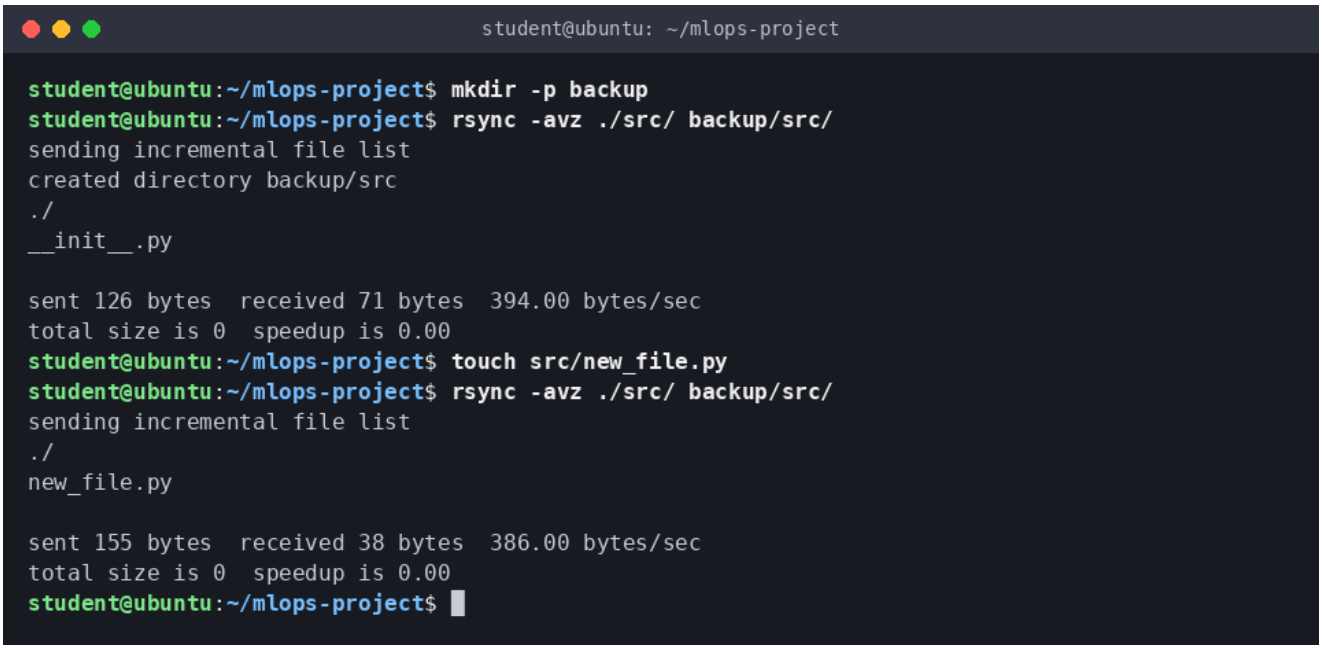
What happened:

- Install them if missing with **sudo apt install zip unzip**.

rsync : smart synchronisation

Copies only what changed, so repeated transfers of big datasets are much faster than scp. Flags: **-a** archive (keeps everything), **-v** verbose, **-z** compress.

Screenshot 49: rsync: the second run copies only the new file.



What happened:

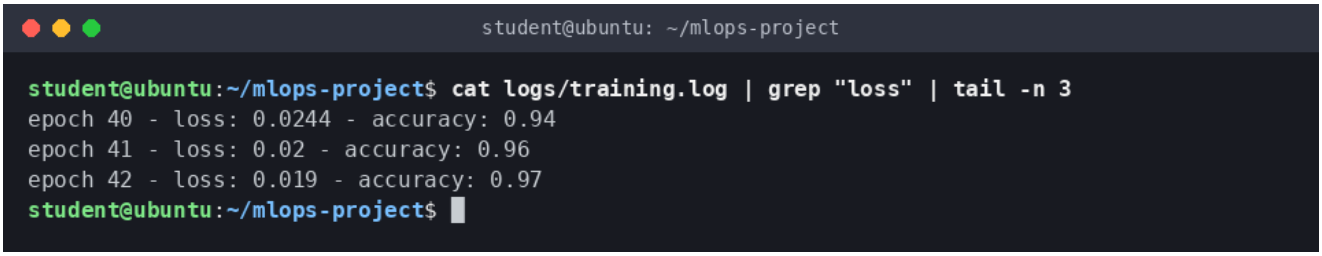
- A trailing slash matters: **./src/** means 'the contents of src'. Remote use: **rsync -avz ./dataset/ user@server:/data/dataset/**.

Step 9: Text Processing and Piping

| : the pipe

Sends the output of one command into the next, so small tools can be chained like a pipeline.

Screenshot 50: cat, grep and tail chained together.

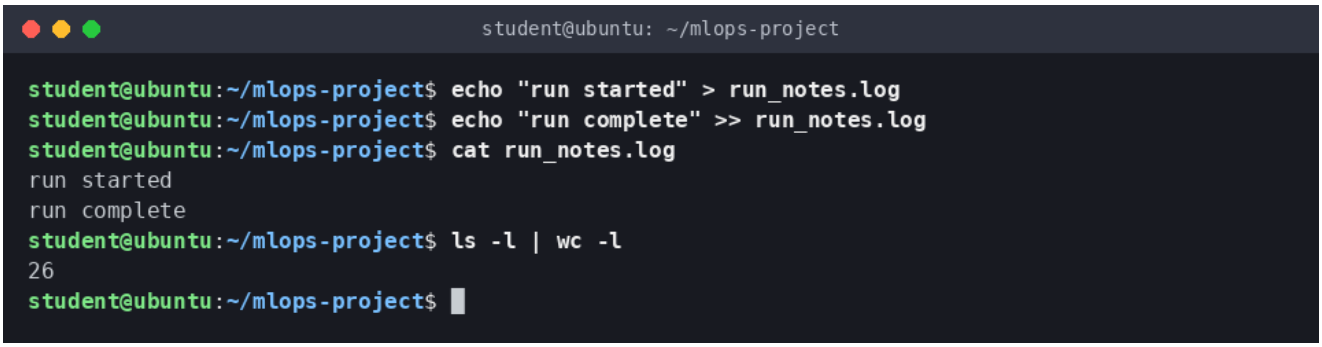


What happened:

- Read it left to right: show the file, keep lines with 'loss', keep the last 3.

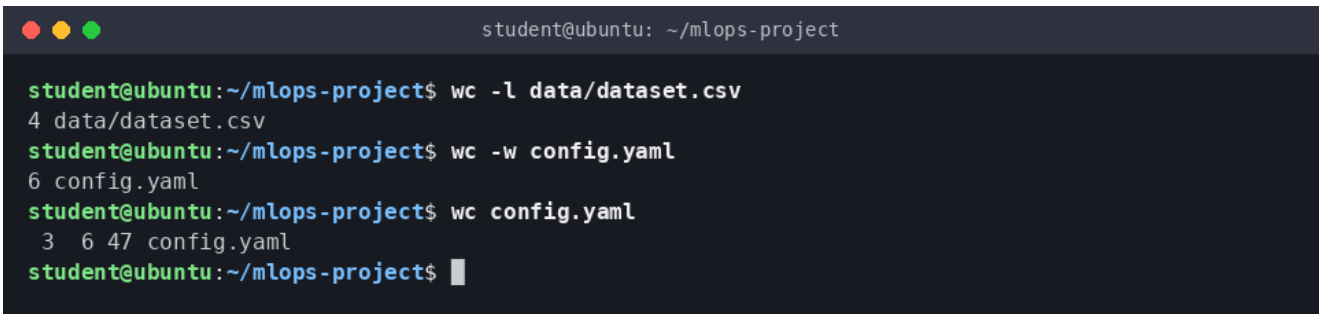
> and >> : redirection

Screenshot 51: > overwrites, >> appends.



wc : count lines, words, characters

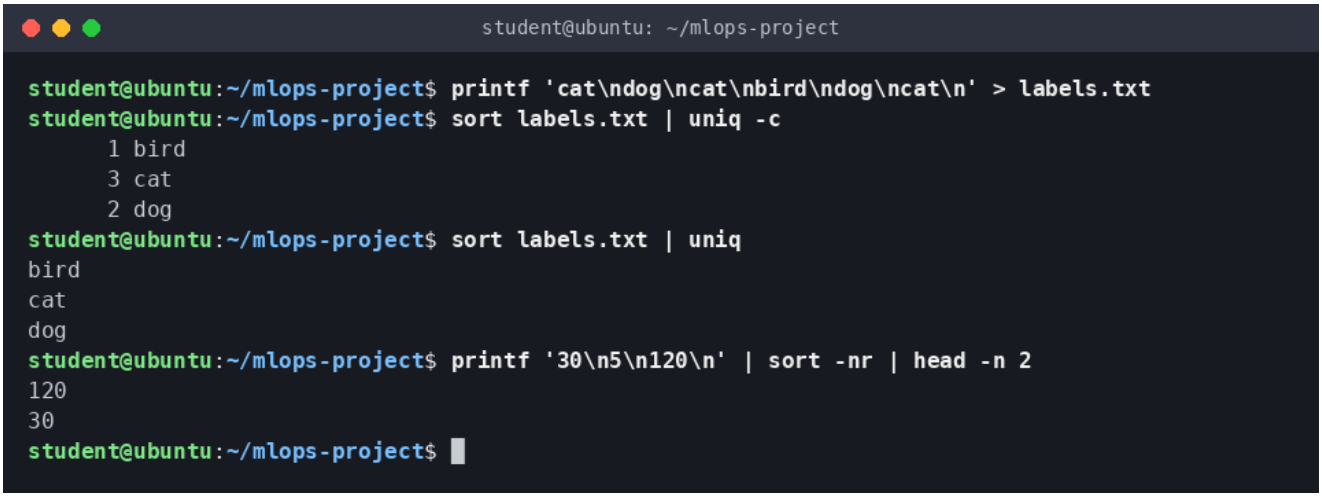
Screenshot 52: wc -l counts rows without opening the file.



sort and uniq

sort orders lines (**-n** numeric, **-r** reverse). **uniq** removes or counts neighbouring duplicates, so use it after sort.

Screenshot 53: Counting how often each label appears.



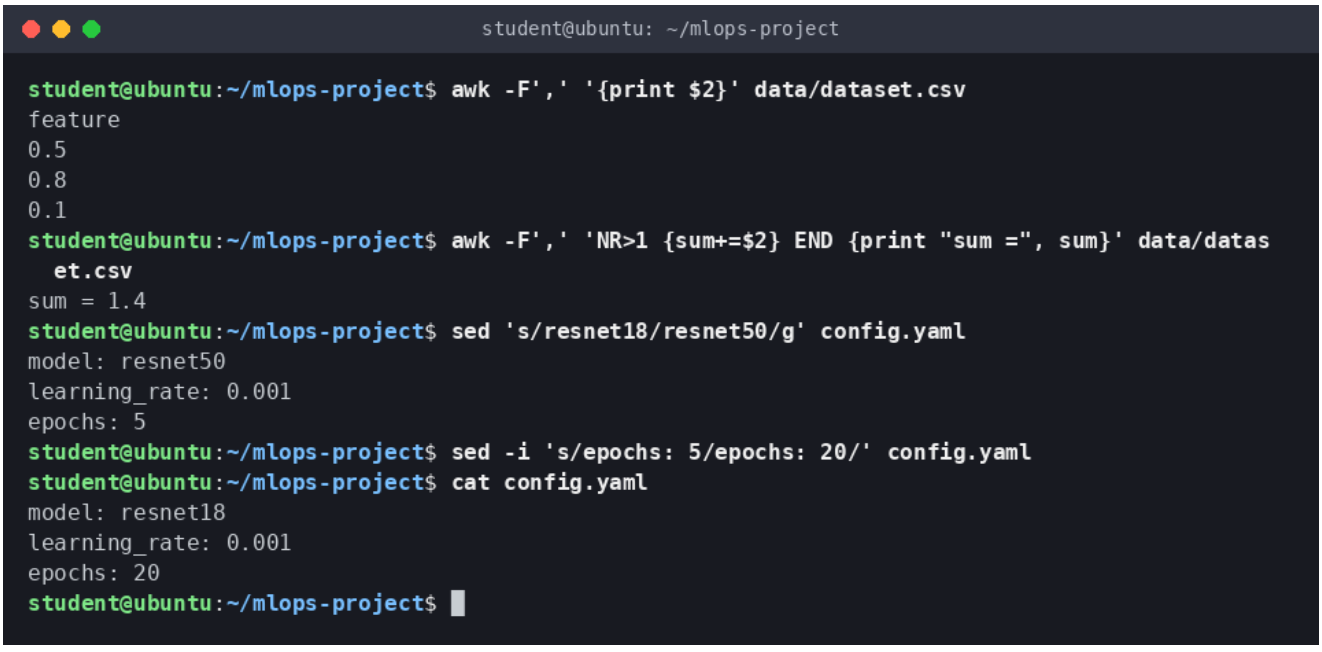
What happened:

- **uniq -c** puts the count in front: this is how you quickly check class balance of a dataset.

awk and sed

awk works column by column; **-F','** says columns are separated by commas, and **{print $2}** prints the second column. **sed** edits text; **s/old/new/g** replaces every 'old' with 'new'.

Screenshot 54: awk prints a column; sed replaces text.



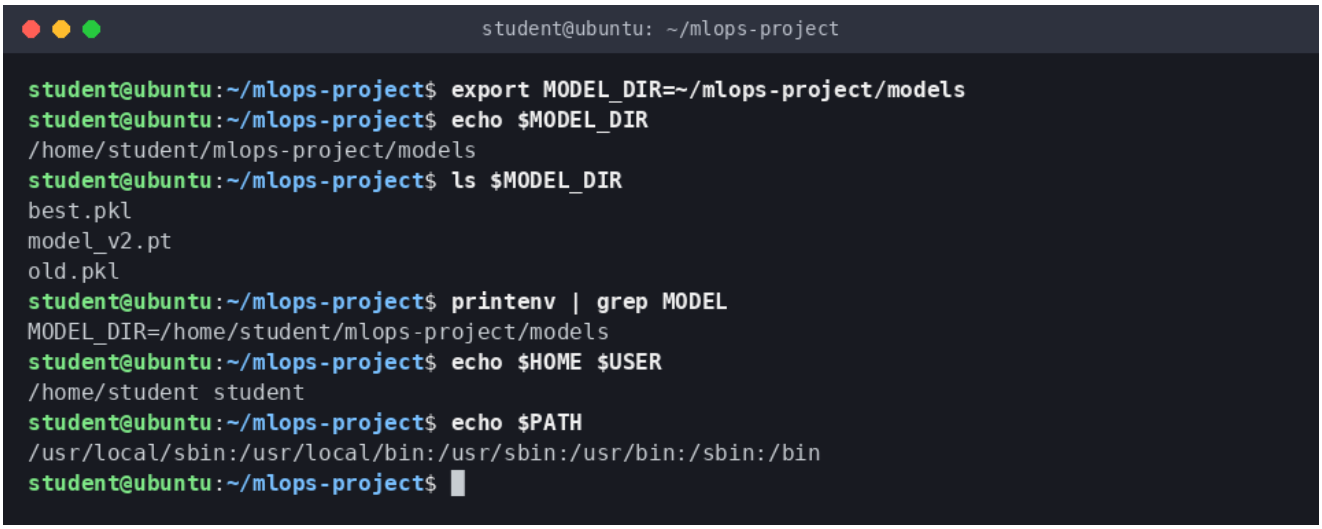
What happened:

- Plain **sed** only shows the changed text. With **-i** it edits the file itself (in place). Be careful with -i.

Step 10: Environment Variables, Python and Packages

An **environment variable** is a named setting that programs can read, such as the data folder or an API key. **export NAME=value** sets one; **$NAME** reads it.

Screenshot 55: Setting and using environment variables.



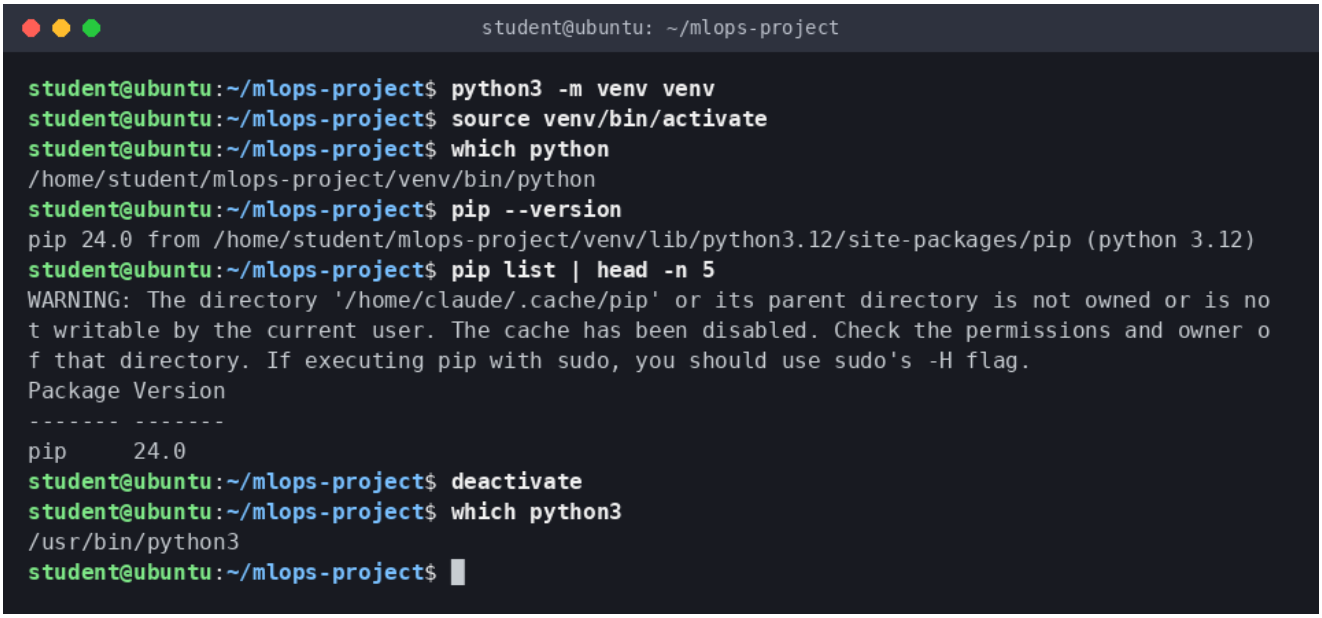
What happened:

- **export** lasts only for this terminal session. To keep it forever, add the line to **~/.bashrc**.
- **PATH** is the list of folders Linux searches when you type a command name.

Python virtual environment and pip

A virtual environment gives each project its own set of Python packages. This is the standard way to avoid version clashes.

Screenshot 56: Create, activate and leave a virtual environment.



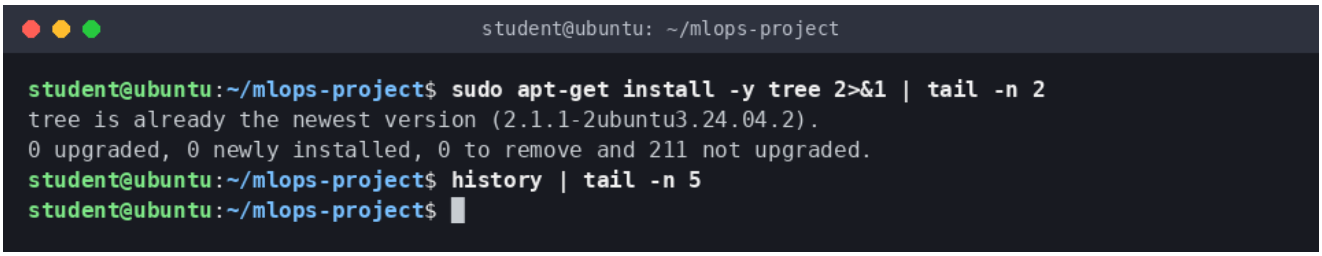
What happened:

- After **source venv/bin/activate** the prompt gets a **(venv)** prefix on a normal terminal, and **which python** points inside the project. Install packages with **pip install numpy** and freeze them with **pip freeze >**

requirements.txt.

- **deactivate** leaves the environment.

Screenshot 57: Installing system packages with apt and viewing command history.



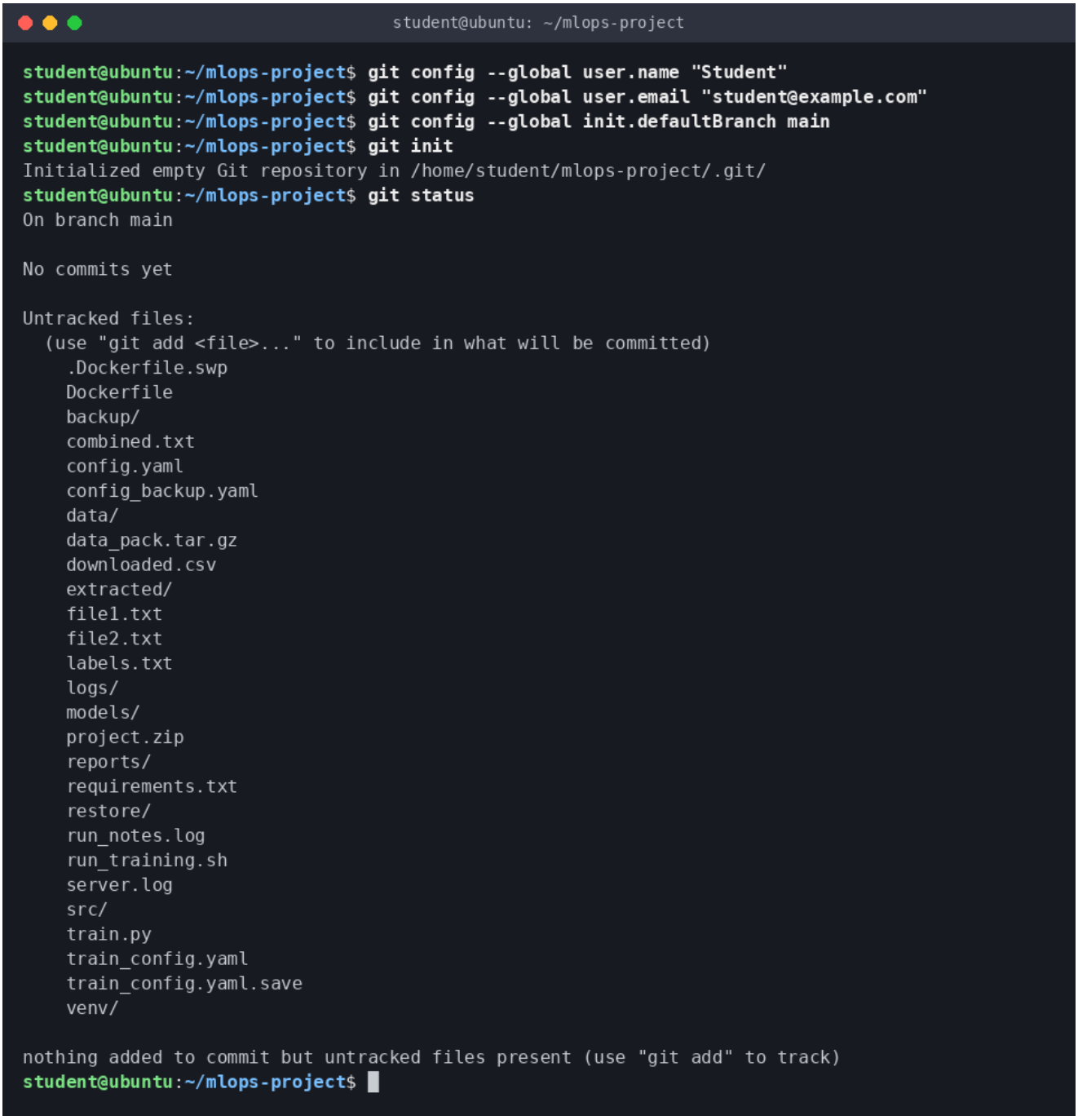
What happened:

- **history** lists your recent commands. Use **!!** to repeat the last command, **Ctrl + R** to search history.

Step 11: Git Basics for MLOps

Git keeps the history of your code. Every MLOps team uses it, and pushing to GitHub/GitLab often triggers the CI/CD pipeline. First tell Git who you are (once per machine):

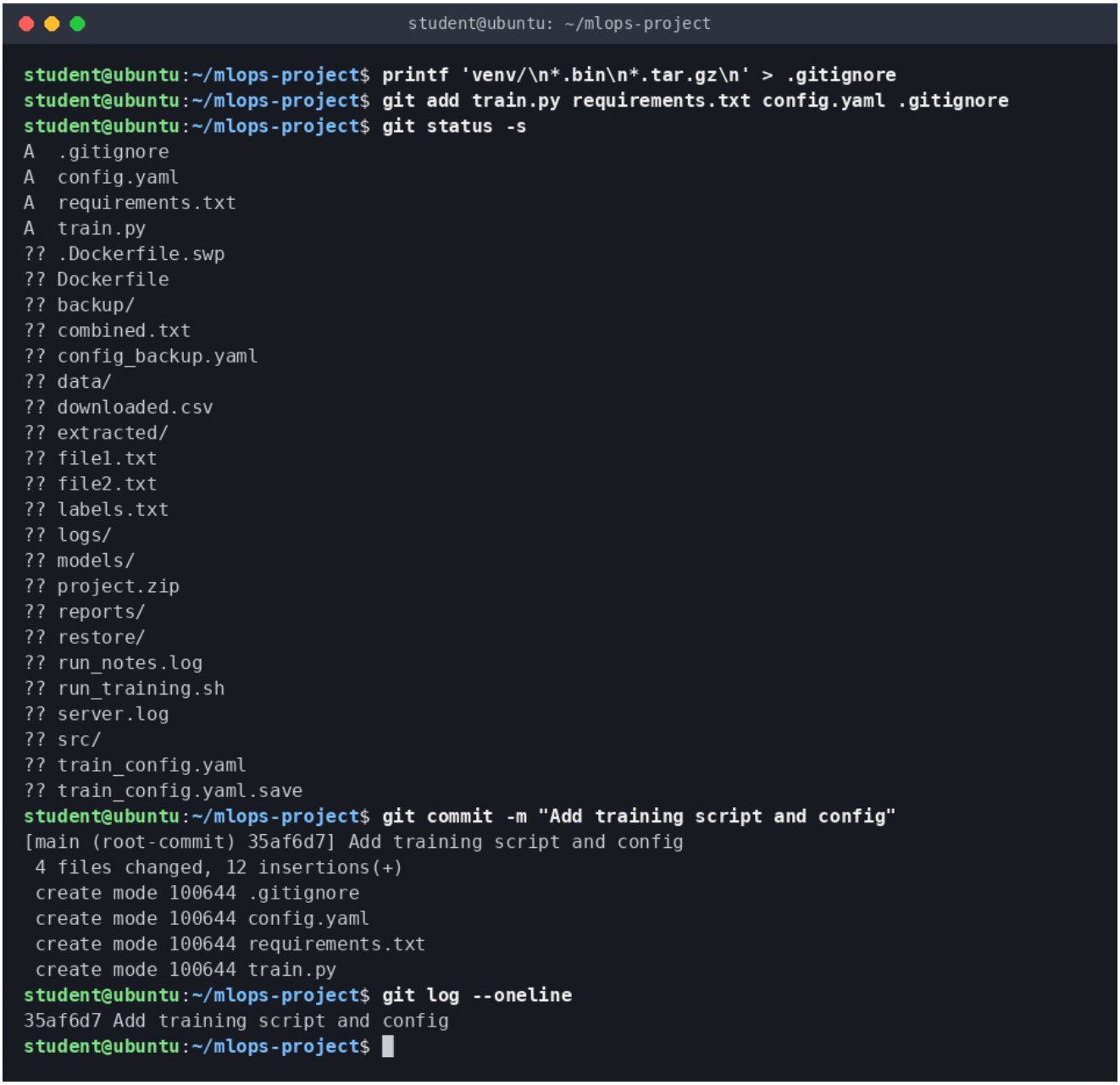
Screenshot 58: git init starts tracking this folder; git status shows what is untracked.



What happened:

- Red/untracked means Git sees the files but does not track them yet.

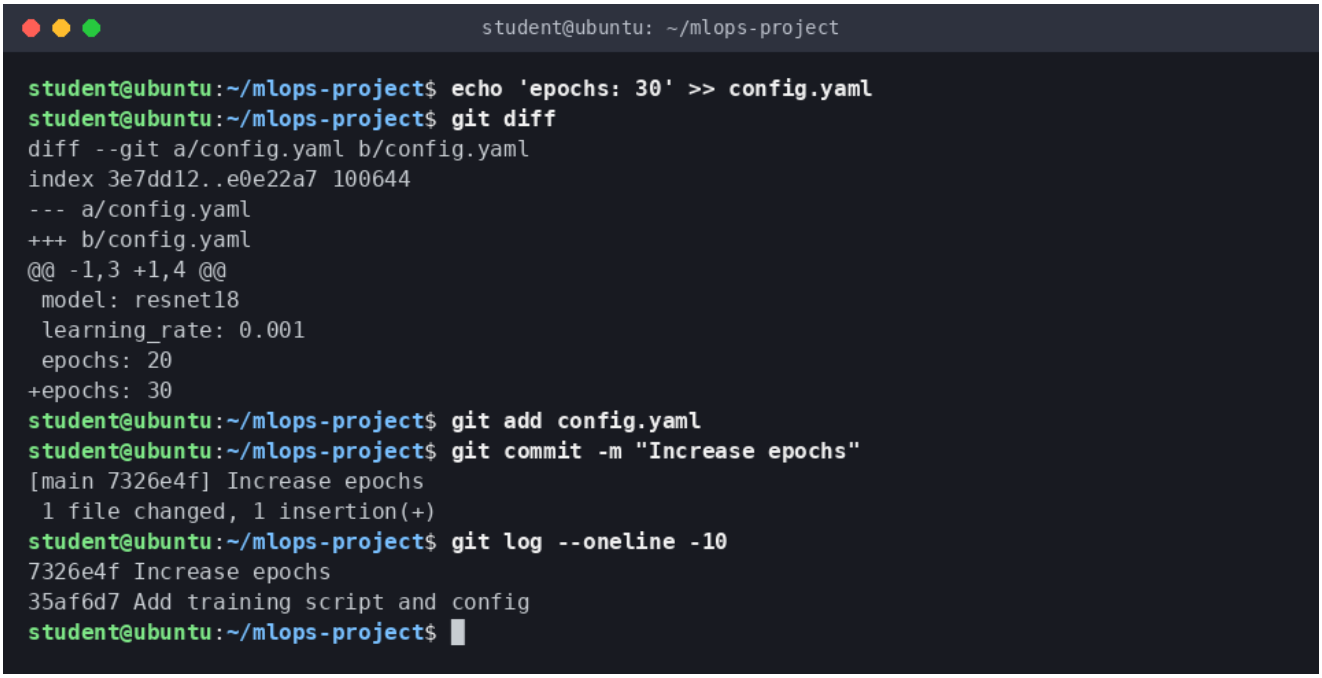
Screenshot 59: git add stages files; git commit saves a snapshot with a message.



What happened:

- **.gitignore** lists files Git must ignore (virtual env, huge data). **git status -s** is the short form: A = added.
- **git log --oneline** shows history, one line per commit.

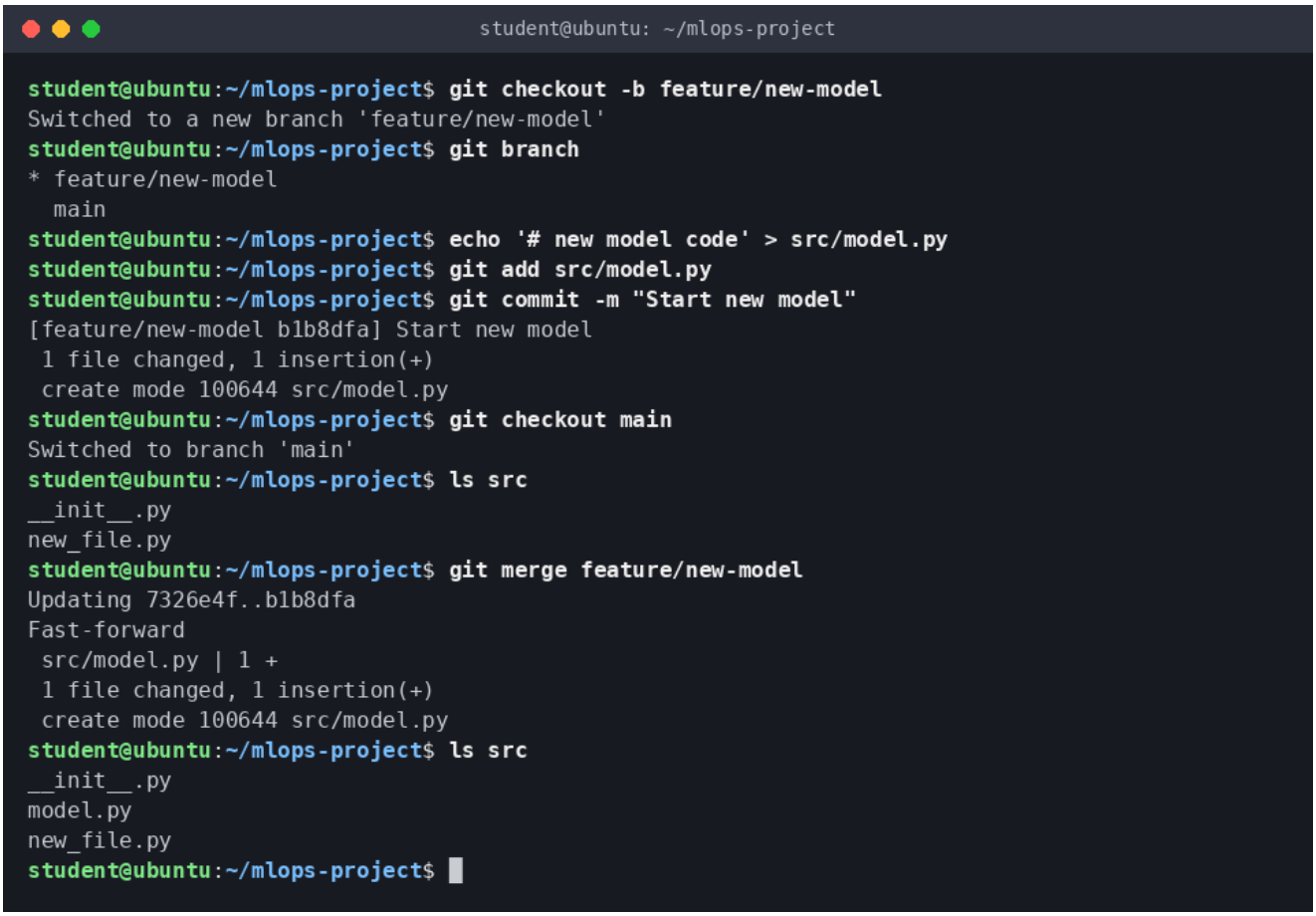
Screenshot 60: git diff shows what changed before you commit.



What happened:

- Lines starting with **+** were added, **-** were removed.

Screenshot 61: Branches: work on a feature safely, then merge.

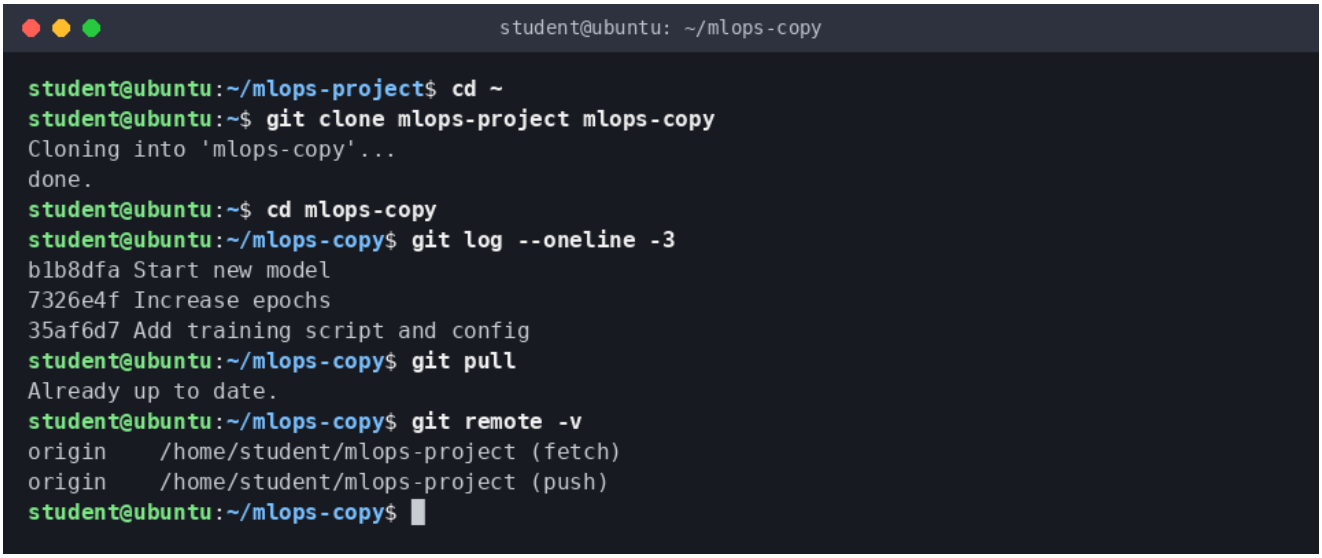


What happened:

- On branch **main** the file model.py does not exist; it only existed on the feature branch until we merged it. **git branch** marks your current branch with *.

git clone, pull, push

Screenshot 62: git clone copies a repository (here from a local folder; normally from GitHub).



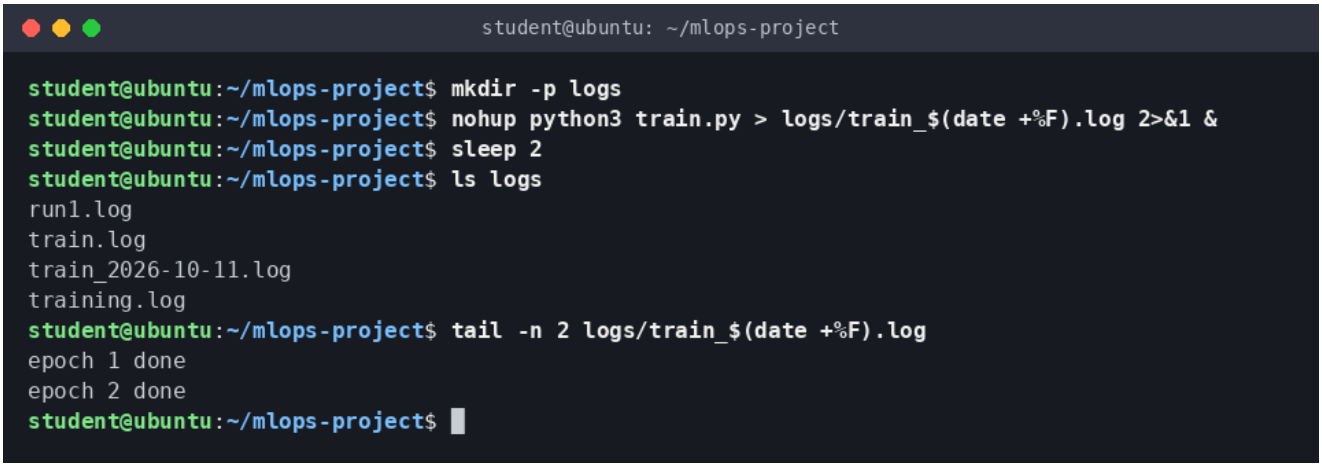
What happened:

- Normally you write **git clone https://github.com/user/project.git**. **git pull origin main** downloads new changes, **git push origin main** uploads yours. Push needs a GitHub account, so it cannot be demonstrated here.

Step 12: Useful MLOps Command Patterns

Run a training job in the background with a dated log

Screenshot 63: nohup + background + date-stamped log file.



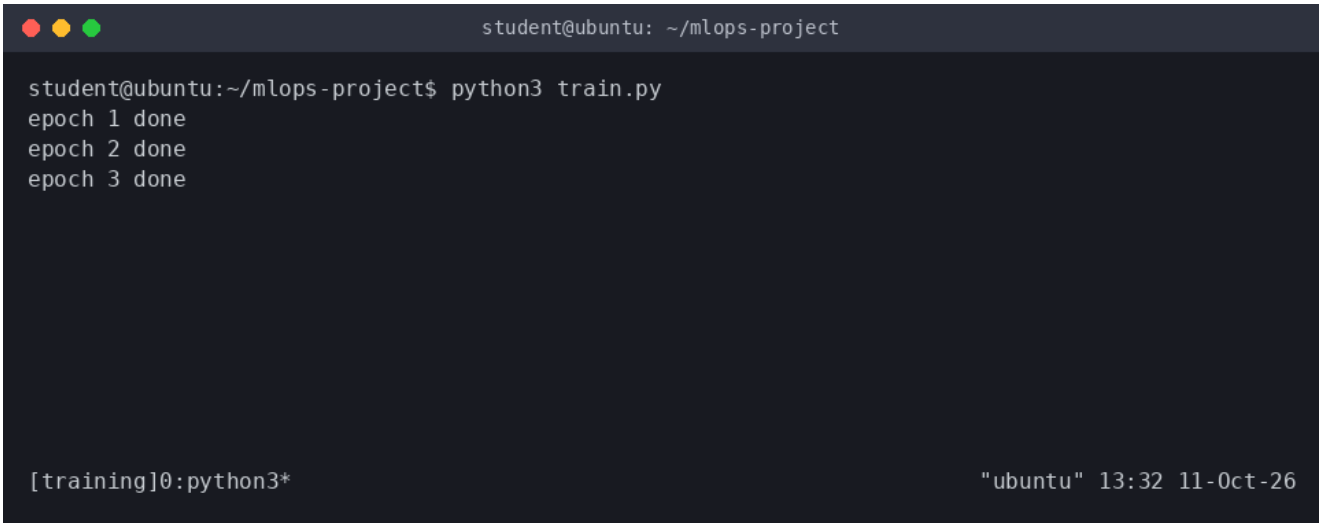
What happened:

- **$(date +%F)** is replaced by today's date such as 2026-10-11, so every run gets its own log file. The job survives a lost SSH connection.

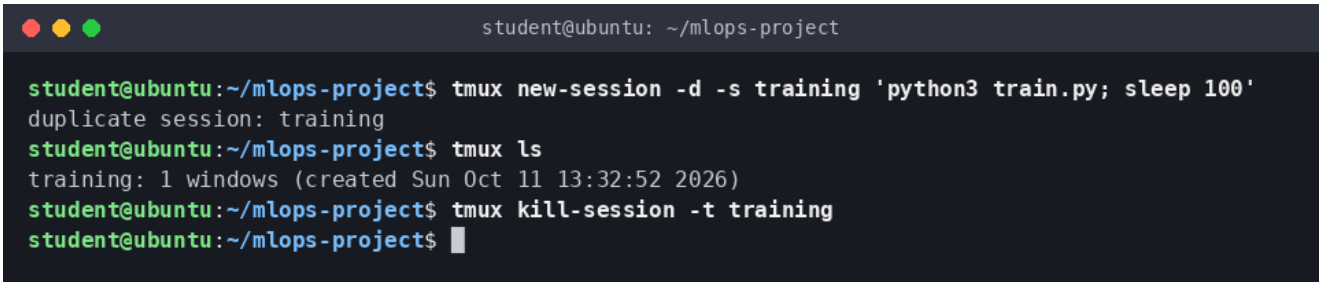
tmux : a terminal session that never dies

tmux creates a session that keeps running on the server even after you disconnect. Start: **tmux new -s training**. Detach: press **Ctrl + B**, release, then **D**. Come back: **tmux attach -t training**. List: **tmux ls**.

Screenshot 64: Inside a tmux session (note the status bar at the very bottom) running the training script.



Screenshot 65: A detached tmux session listed by tmux ls.

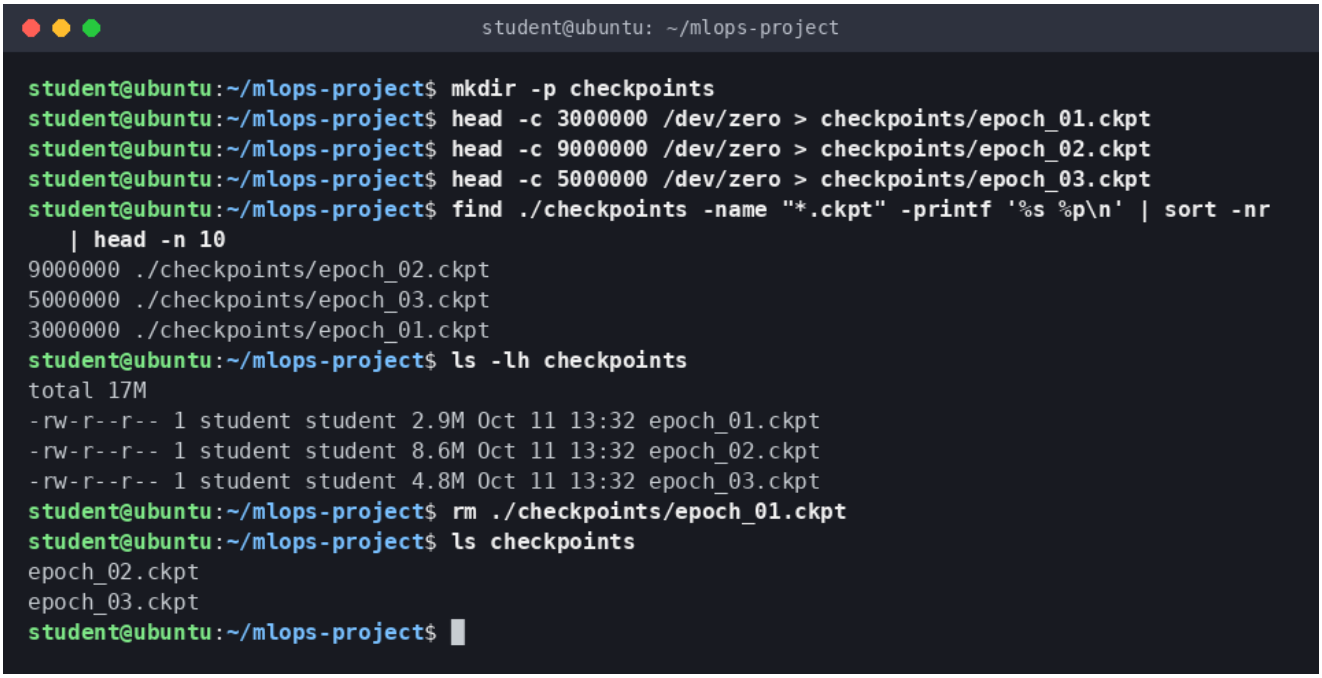


What happened:

- The session keeps running in the background. Reattach with **tmux attach -t training**.

Find and clean up large checkpoint files

Screenshot 66: Largest files first, then remove one that is no longer needed.

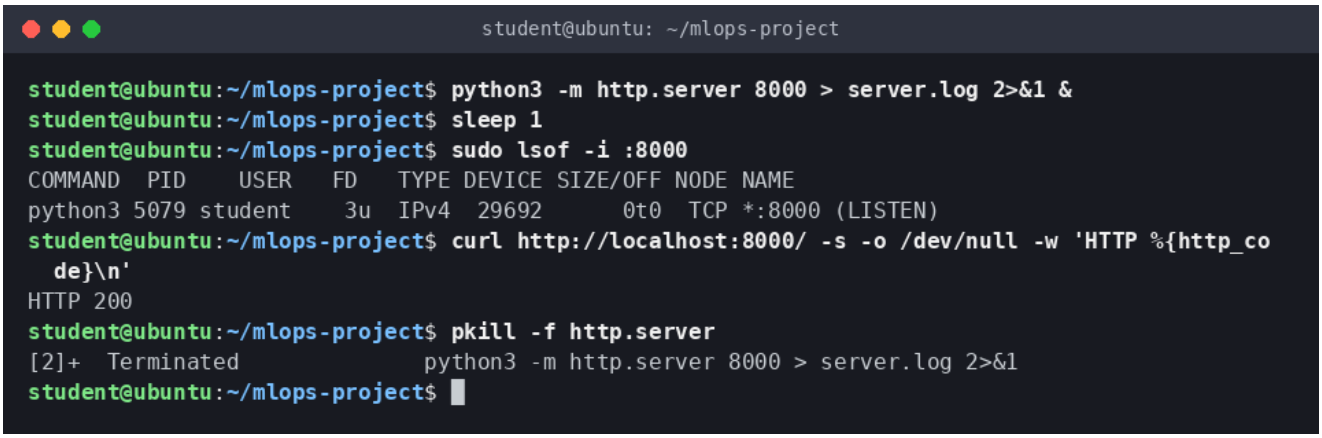


What happened:

- **%s** prints the size in bytes and **%p** the path. **sort -nr** puts the biggest first.

Check that a model service is up

Screenshot 67: lsof + curl health check.



> **TIP:** Run **man grep** (or any command name) to read its full manual. Press q to quit.

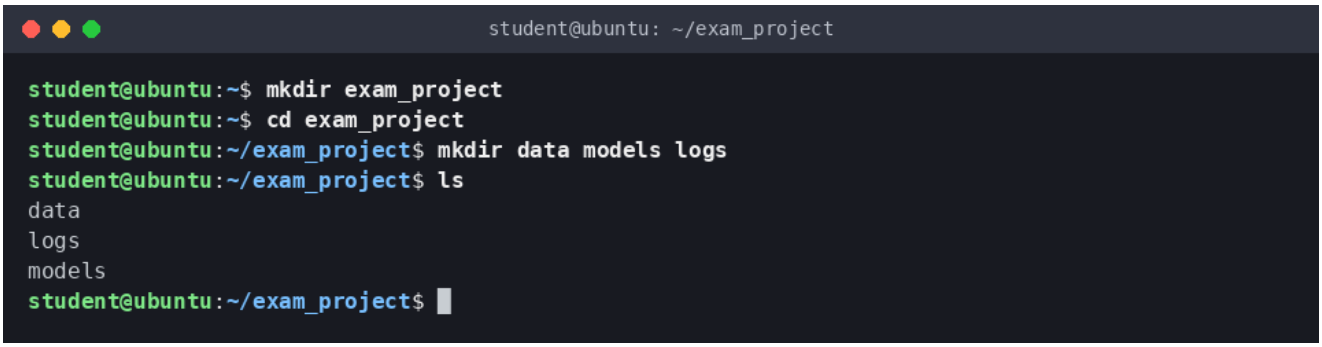
Step 13: Practice Exam. Do It Yourself

Type everything below on your own terminal, then take screenshots. This is the kind of task a practical exam asks. A full worked solution follows with real screenshots.

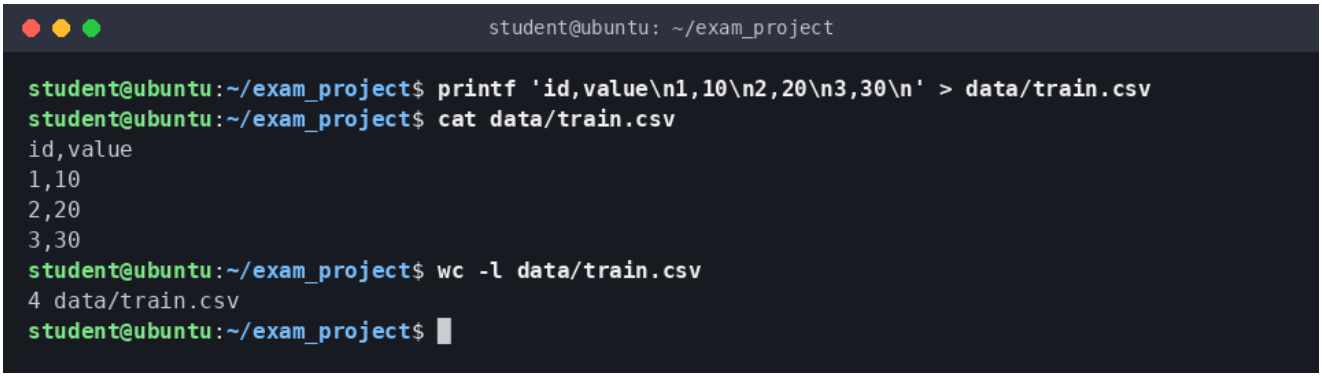
- Create a folder **exam_project** in your home folder and go inside it.
- Inside it create folders **data**, **models**, **logs** in one command.
- Create **data/train.csv** with 4 lines of text and show it.
- Count the lines of the CSV file.
- Create a script **run.sh** that prints 'Training started', make it executable and run it, saving output in **logs/run.log**.
- Show the permissions of run.sh.
- Search the log for the word 'Training'.
- Copy **data** to **data_backup**, then compress the project to a tar.gz.
- Show disk usage of the project and free memory.
- Initialise git and make one commit.
- Delete the backup folder.

Worked solution

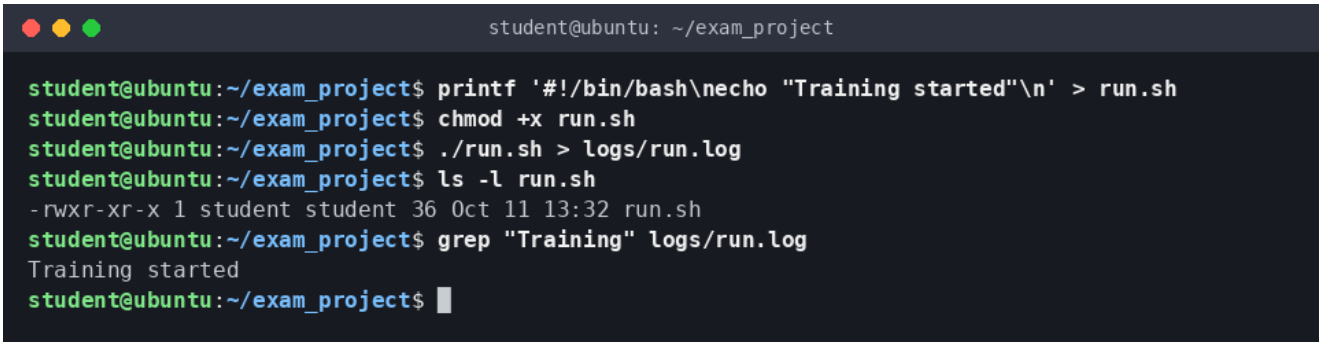
Screenshot 68: Task 1 and 2: folders created.



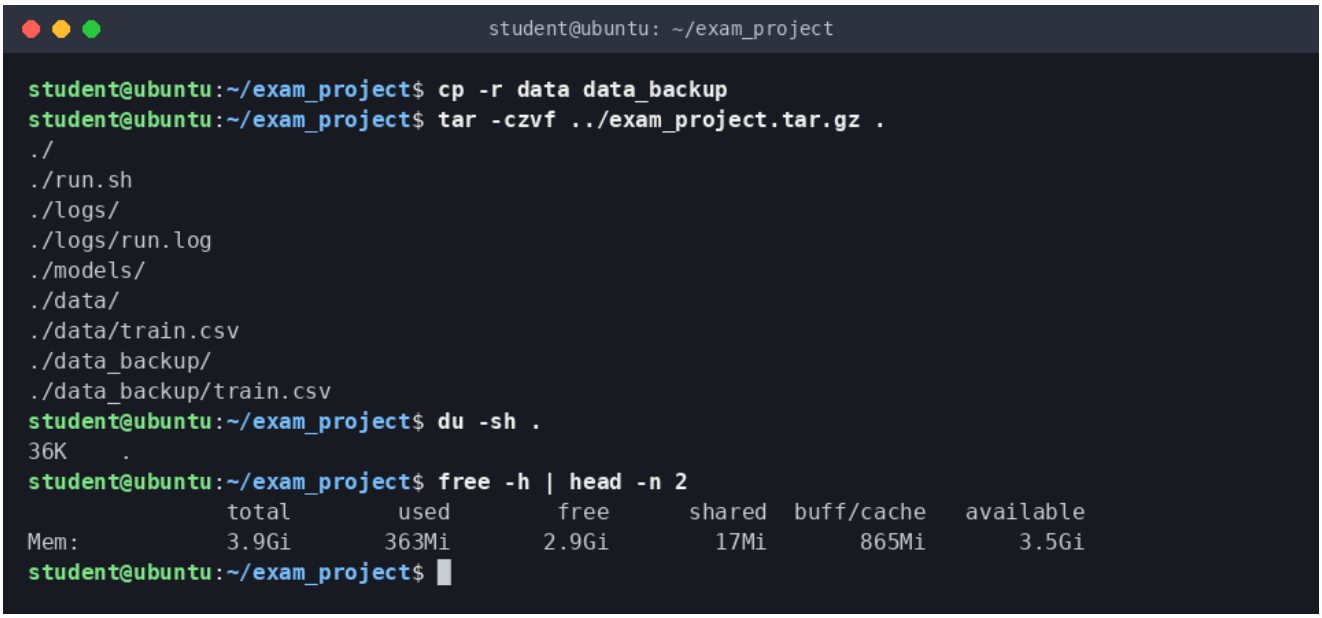
Screenshot 69: Task 3 and 4: create, show and count a file.



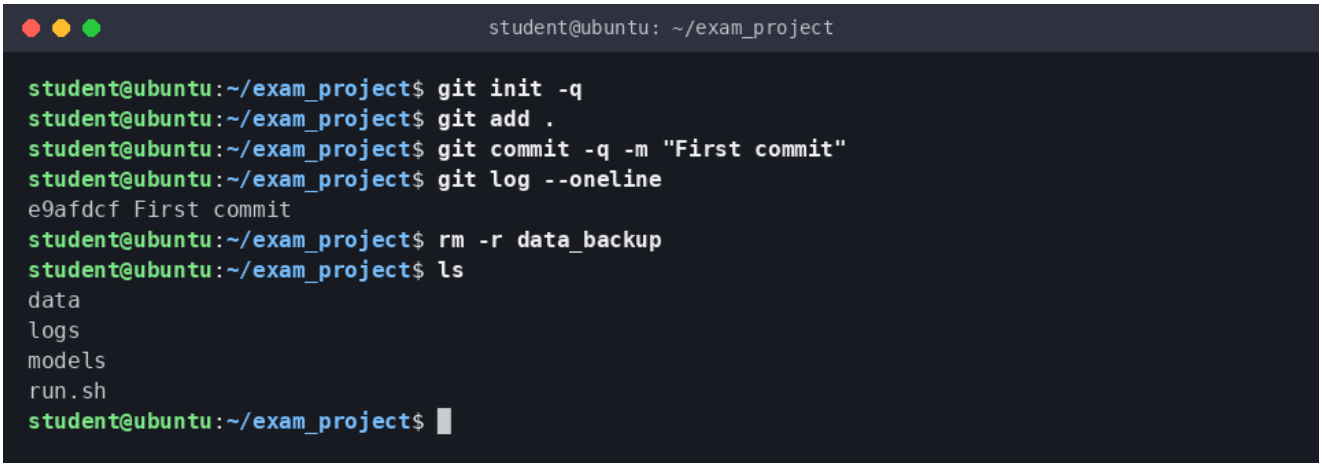
Screenshot 70: Task 5, 6 and 7: script, permission, log and search.



Screenshot 71: Task 8 and 9: copy, compress, disk and memory.



Screenshot 72: Task 10 and 11: git commit and cleanup.



Cheat Sheet: Every Command on One Page

Command	What it does	Example
pwd	show current folder	pwd
ls	list files	ls -lah
cd	change folder	cd .. cd ~ cd -
find	search files	find . -name "*.pkl"
which	path of a command	which python3
mkdir	make folder	mkdir -p a/b
touch	create empty file	touch a.txt
cp	copy	cp -r src dst
mv	move / rename	mv a.txt b.txt
rm	delete (permanent)	rm -r folder
cat	show file	cat a.txt
less	page through	less log (q quits)
head / tail	start / end of file	tail -f log
nano / vim	edit file	nano a.txt
grep	search text	grep -rn "word" .
chmod	permissions	chmod +x run.sh
chown	owner	sudo chown u:u file
sudo	run as admin	sudo apt update
ps / top	processes	ps aux | grep python
kill	stop a process	kill PID kill -9 PID
& nohup	background job	nohup cmd > log 2>&1 &
jobs fg bg	manage jobs	jobs fg %1
df -h	free disk	df -h
du -sh	folder size	du -sh data
free -h	memory	free -h
nvidia-smi	GPU status	nvidia-smi
watch	repeat command	watch -n 2 df -h
ssh / scp	remote login / copy	ssh user@host
curl / wget	web request / download	curl -I URL
ping	test network	ping -c 4 host
tar	archive	tar -czvf a.tar.gz dir / tar -xzvf a.tar.gz
zip / unzip	zip files	zip -r a.zip dir
rsync	sync folders	rsync -avz src/ dst/

Command	What it does	Example
| > >>	pipe / write / append	cmd | grep x > out.txt
wc / sort / uniq	count / order / dedupe	sort f | uniq -c
awk / sed	columns / replace	sed -i 's/a/b/g' f
export	set variable	export NAME=value
git	version control	git add . && git commit -m msg
tmux	persistent session	tmux new -s name

Common errors and what they mean

Message	Reason and fix
command not found	typo, or program not installed. Check spelling, or sudo apt install name
No such file or directory	wrong name/path or wrong upper/lower case. Check with pwd and ls
Permission denied	no right to read/write/run. Use chmod +x for scripts, or sudo
Is a directory	you used rm/cp on a folder: add -r
File exists	the name is already used. Pick another name
Terminal looks frozen	you are probably inside less/vim/nano/top. Try q, or Esc then :q!, or Ctrl + C

Exam day checklist

- Open the terminal (Ctrl + Alt + T) and run **pwd** first so you know where you are.
- Type commands slowly. Use Tab for auto-complete and the Up arrow to repeat.
- After EVERY task, take a screenshot that shows the command AND its output. Windows: Win + Shift + S. Ubuntu: Print Screen (or Shift + Print Screen to choose an area).
- Keep names simple: lowercase, no spaces.
- If something fails, read the error message, fix the one thing it mentions, and try again.
- Before using rm -r or rm -rf, run ls on the same path.